
CiVerLy

Release 1.0.0

cryptosolutions GmbH

Oct 15, 2025

CONTENTS

1	Install	3
2	User Manual	9
3	Source Code Documentation	27
4	Developer Manual	67
	Python Module Index	71
	Index	73

This is the reference manual for CiVerLy. Here, you find installation instructions, a manual for users and developers and the documentation of the source code.

INSTALL

1.1 Installation of Dependencies

1.1.1 Install SageMath

We install sage in Version 10.4. First, install all the prerequisites, build tools and LATEX related tools for sage. For Debian/Ubuntu, install those by running the following commands. For other Linux systems, consult the [sage installation manual](#).

```
$ sudo apt-get install bc binutils bzip2 ca-certificates cliquer cmake curl \
    ecl eclib-tools fflas-ffpack g++ gap gcc gengetopt gfan gfortran \
    glpk-utils gmp-ecm lcalc libatomic-ops-dev libboost-dev \
    libbraiding-dev libbrial-dev libbrial-groebner-dev libbz2-dev \
    libcdd-dev libcdd-tools libcliquer-dev libcurl4-openssl-dev libec-dev \
    libecm-dev libffi-dev libflint-dev libfp11l-dev libfreetype-dev \
    libgap-dev libgc-dev libgd-dev libgf2x-dev libgiac-dev libgivaro-dev \
    libglpk-dev libgmp-dev libgsl-dev libhomfly-dev libiml-dev \
    liblfunction-dev liblinbox-dev liblrcalc-dev liblzma-dev libm4ri-dev \
    libm4rie-dev libmpc-dev libmpfi-dev libmpfr-dev libncurses5-dev \
    libntl-dev libopenblas-dev libpari-dev libplanarity-dev libppl-dev \
    libprimesieve-dev libpython3-dev libqhull-dev libreadline-dev \
    librw-dev libsingular4-dev libsqlite3-dev libssl-dev \
    libsuitesparse-dev libsymmetrica2-dev libz-dev libzmq3-dev m4 make \
    maxima maxima-sage meson nauty ninja-build openssl palp pari-doc \
    pari-elldata pari-galdata pari-galpol pari-gp2c pari-seadata patch \
    patchelf perl pkg-config planarity ppl-dev python3 python3-setuptools \
    python3-venv singular singular-doc sqlite3 sympow tachyon tar texinfo \
    tox xcas xz-utils
$ sudo apt-get install autoconf automake gh git gpgconf libtool \
    openssl-client pkg-config
$ sudo apt-get install default-jdk dvipng ffmpeg fonts-freefont-otf \
    imagemagick latexmk libavdevice-dev libjpeg-dev pandoc tex-gyre \
    texlive-fonts-recommended texlive-lang-cyrillic texlive-lang-english \
    texlive-lang-european texlive-lang-french texlive-lang-german \
    texlive-lang-italian texlive-lang-japanese texlive-lang-polish \
    texlive-lang-portuguese texlive-lang-spanish texlive-latex-extra \
    texlive-luatex texlive-xetex xindy
```

Next, obtain the sources for sage by running the following commands:

```
$ cd ~
$ git clone --origin upstream https://github.com/sagemath/sage.git
$ cd sage
$ git checkout 10.4
```

Now, build sage by running the following commands:

```
$ make configure
$ export MAKE="make -j4"
$ ./configure --with-system-python3=no
$ make
```

Note: The command `export MAKE="make -j4"` instructs the build process to run four jobs in parallel. If your CPU is sufficiently strong, you might be able to speed up the build process by increasing this number, e.g., to eight or even 16. On the other hand, if the build process fails, try to run again without this command.

Finally, make sage available globally:

```
$ sudo ln -s $PWD/sage /usr/local/bin/sage
```

Start sage (and then exit it again) to verify that the correct version is installed:

```
$ sage
+-----+
| SageMath version 10.4, Release Date: 2024-07-19          |
| Using Python 3.12.4. Type "help()" for help.             |
+-----+
sage: exit()
```

1.1.2 Install GLPK

We use GLPK, the GNU Linear Programming Kit, as MILP solver. To install GLPK (Version 5.0), first download the sources by running the following commands:

```
$ cd ~
$ wget https://ftp.gnu.org/gnu/glpk/glpk-5.0.tar.gz
$ tar -vxf glpk-5.0.tar.gz
```

Then, install GLPK by running the following commands:

```
$ cd glpk-5.0
$ ./configure
$ make
$ sudo make install
```


1.1.3 Install SCIP

SCIP is another MILP solver. Run the following commands to install SCIP (Version 9.1.0). Notice that soplex is not available in the package repository of Ubuntu 22.04 (it is for Ubuntu 24.04). If your system is connected to the internet, soplex will be downloaded and installed automatically during the installation of SCIP.

```
$ sudo apt-get install wget cmake g++ m4 xz-utils libgmp-dev unzip zlib1g-dev \
    libboost-program-options-dev libboost-serialization-dev \
    libboost-regex-dev libboost-iostreams-dev libtbb-dev libreadline-dev \
    pkg-config git liblapack-dev libgsl-dev flex bison libcliquer-dev \
    gfortran file dpkg-dev libopenblas-dev rpm soplex libsoplex-dev
$ cd ~
$ git clone https://github.com/scipopt/scip.git
$ cd scip
$ git checkout v910
$ mkdir build
$ cd build
$ cmake .. -DPAPILO=off -DZIMPL=off -DIPOPT=off
$ make
$ make check
$ sudo make install
```

1.1.4 Install Gurobi

Gurobi is commercial MILP solver. That is, a licence is required to use it. Here, we assume that you have already setup an account on <https://www.gurobi.com/> and already have obtained a licence there.

To install Gurobi (Version 11.0.3), go to <https://www.gurobi.com/downloads/gurobi-software/> and download the archive for x64 Linux. We assume that the archive is stored in the ~/Downloads directory. Run the following commands:

```
$ cd ~
$ mv ~/Downloads/gurobi11.0.3_linux64.tar.gz .
$ tar -xzf gurobi11.0.3_linux64.tar.gz
$ rm gurobi11.0.3_linux64.tar.gz
```

In the following, we assume that the used shell is bash. Verify this by running

```
$ echo $0
```

If the result is not bash, adapt the following commands accordingly. To make the Gurobi executables available globally, we have to add some lines to the .bashrc file.

```
$ echo '# Make Gurobi files available globally' >> ~/.bashrc
$ echo 'export GUROBI_HOME=~/.gurobi1103/linux64/' >> ~/.bashrc
$ echo 'export PATH="$GUROBI_HOME/bin:$PATH"' >> ~/.bashrc
$ echo 'export LD_LIBRARY_PATH="{GUROBI_HOME}/lib"' >> ~/.bashrc
$ source ~/.bashrc
```

Finally, goto to <https://portal.gurobi.com/iam/licenses/list> which should list your licence. On the right-hand side, click the icon with a monitor and a downward facing arrow (next to the bell). This will show you a command of the following form:

```
$ grbgetkey xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

Copy it and execute it on your machine.

1.1.5 Install CryptoMiniSat

We use CryptoMiniSat (Version 5.11.21) as SAT solver. We recommend installing CryptoMiniSat from source. For this, we follow the instructions in the Compiling in Linux section on the [CryptoMiniSat GitHub page](#). First, if not yet installed, we need to install some build tools. On Debian/Ubuntu run the following command:

```
$ sudo apt-get install build-essential cmake
```

Next, install cadical using the following commands:

```
$ cd ~
$ git clone https://github.com/meelgroup/cadical
$ cd cadical
$ git checkout mate-only-libraries-1.8.0
$ ./configure
$ make
$ cd ..
```

Now, install cadiback using the following commands:

```
$ git clone https://github.com/meelgroup/cadiback
$ cd cadiback
$ git checkout mate
$ ./configure
$ make
$ cd ..
```

Finally, install CryptoMiniSat:

```
$ git clone https://github.com/msoos/cryptominisat.git
$ cd cryptominisat
$ git checkout 5.11.21
$ mkdir build && cd build
$ cmake ..
$ make
$ sudo make install
$ sudo ldconfig
```

1.1.6 Install Cadical

Cadical is another SAT solver. To install Cadical (Version 2.0.0), run the following commands.

```
$ cd ~
$ git clone https://github.com/arminbiere/cadical cadical-standalone
$ cd cadical-standalone
$ git checkout rel-2.0.0
$ ./configure
$ make
```

Finally, make Cadical available globally:

```
$ sudo ln -s $PWD/build/cadical /usr/local/bin/cadical
```

1.1.7 Install Espresso

Espresso is a logic minimizer. To install Espresso (Version 2.3), run the following commands.

```
$ cd ~
$ git clone https://github.com/classabbyamp/espresso-logic.git
$ cd espresso-logic
$ git checkout 1.1
$ cd espresso-src
$ make
```

Finally, make Espresso available globally:

```
$ cd ../bin
$ sudo ln -s $PWD/espresso /usr/local/bin/espresso
```

1.2 Installation of CiVerLy

Make sure that the CiVerLy environment, especially SageMath, is set up as described before. As all dependencies of CiVerLy are already installed within SageMath, which can be verified by running

```
$ sage --python -m pip list packages
```

and making sure that `setuptools` and `wheel` are in the list, an installation without internet connection is no problem. Install CiVerLy by running:

```
$ make
$ sage --python -m pip install --no-index --no-build-isolation .
```

The `--no-index` flag will stop `pip` from trying to search for the dependencies online and the `--no-build-isolation` flag allows `pip` to use the already installed versions of the dependencies.

1.2.1 Testing CiVerLy (Optional)

To run the tests, change to the packages root directory `civerly` and run the following command, but **remove solvers that are not installed**:

```
$ sage -t --optional=sage,scip,glpk,gurobi,cryptominisat,cadical,espresso src/civerly
```

Warning: The doctests temporarily add and remove files while testing. Therefore it is advised to not run several doctests simultaneously, in order to avoid concurrency issues!

1.3 Installation via Docker

Alternatively, you can install CiVerLy and its dependencies via docker. To build the docker image run the following command in the civerly directory. Notice that this will take some time.

```
$ docker build -t civerly -f docker/Dockerfile .
```

To start a sage shell inside the container run:

```
$ docker run -it civerly
```

To start a bash shell inside the container run:

```
$ docker run -it --entrypoint bash civerly
```

To share the docker image, save it to a file:

```
$ docker save civerly > civerly.tar
```

This file can then be loaded again (on a different machine) with:

```
$ docker load < civerly.tar
```

2.1 The CiVerLy Workflow

We first describe the CiVerLy workflow and the associated interfaces which we visualize below.

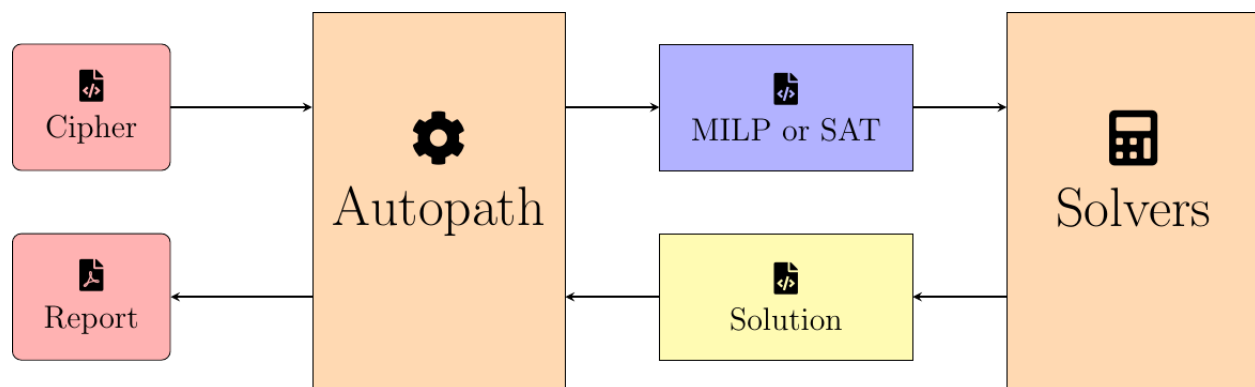


Fig. 1: The CiVerLy workflow.

In a nutshell, there are four steps. We describe them briefly below and give more details on the respective pages. For examples showcasing this, we refer to the ciphers already implemented in CiVerLy, especially the AES and the corresponding examples that show how to use CiVerLy to generate models for differential and linear cryptanalysis of AES.

Implementing the Cipher

The first step is to implement the cipher inside of CiVerLy. The implementation has the form of a directed acyclic graph which consists of components that come with CiVerLy. For more details, see [Implementing the Cipher](#).

Modeling the Cipher

Once you have implemented the cipher, you can model it. For this, you have to choose a model. There are different models, varying in the complexity to solve them and the accuracy of the results. For more details, see [Modeling the Cipher](#).

Solving the Model

Once you have the model, you need to solve it. This is done with an external solver which might run on a different machine. If CiVerLy and the solver are installed on the same machine, you can start the solver from within CiVerLy. For more details, see [Solving the Model](#).

Generating the Report

The final step is to generate the report. This can be as simple as printing e.g. the number of active S-boxes but you can also generate a PDF containing the details of the found trail. For more details, see [Generating the Report](#).

2.1.1 Implementing the Cipher

In CiVerLy, ciphers are implemented as Python (or more specifically sage) objects. As different kinds of ciphers are suited differently well for the various modeling options, there is not only one cipher class in CiVerLy but a hierarchy as depicted below.

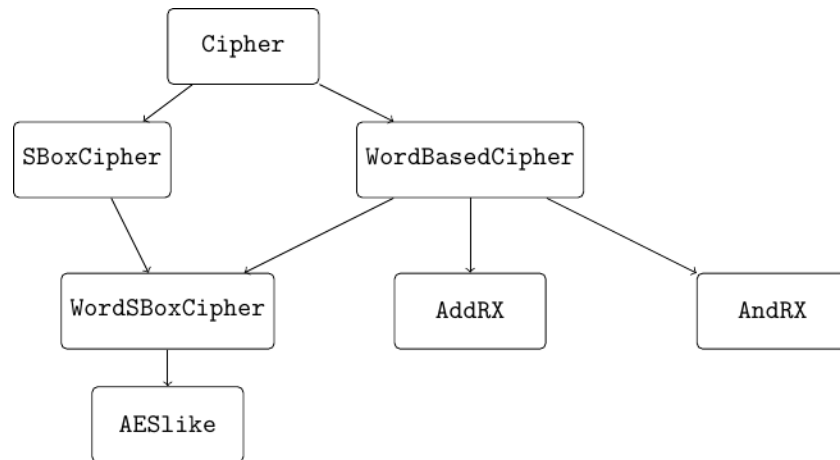


Fig. 2: The class hierarchy of CiVerLy ciphers.

The `Cipher` class is the base class for all ciphers in CiVerLy. It further supports the use of all components implemented in CiVerLy. However this versatility comes with the drawback that modeling `Cipher` objects with MILP is not supported, as the components performing modular addition and logical AND (`ModAdd_CVL` and `AND_CVL`, respectively) can only be modeled using SAT. This is demonstrated with the code below.

```

# Four S-boxes in parallel followed by modular addition
from civerly.component import *
from civerly.cipher import Cipher
from sage.crypto.sboxes import PRESENT
ciph = Cipher(input_length=16, output_length=8, name="ciph")
sb = SBox_CVL(PRESENT)
modadd = ModAdd_CVL(word_length=8)
nodes = []
for j in range(4):
    nodes.append(
        ciph.add_subcipher(sb, [(ciph.IN, (i + 4*j, i)) for i in range(4)])
    )
node_out = ciph.add_subcipher(
    modadd, [(nodes[i // 4], (i % 4, i)) for i in range(16)]
)
ciph.add_output([(node_out, (i, i)) for i in range(8)])

# Modeling with MILP is not possible
from civerly.model_options import *
model_options = MODEL_OPTIONS(
    cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,

```

(continues on next page)

(continued from previous page)

```

    optimization=OPTIMIZATION.MILP,
    granularity=GRANULARITY.BITWISE,
    sbbox_modeling=SBOX_MODELING.CONVEX_HULL,
    solver=SOLVER.GLPK
)
ciph.model(model_options)
# Causes an error:
# NotImplementedError: MILP modeling is not supported for
# <class 'civerly.cipher.Cipher'>!
```

Next, we differentiate between ciphers containing only S-boxes as non-linear components, which are captured by the `SBoxCipher` class and its subclasses, respectively. We want to emphasize again that in contrast to SAT modeling, MILP modeling is only supported for instances of the `SBoxCipher` class and the classes inheriting from it, since `SBox_CVL` is the only non-linear component that may be modeled using MILP in CiVerLy. Adding components which are not allowed causes an error directly when executing `add_subcipher`:

```

from civerly.component import *
from civerly.sboxcipher import SBoxCipher
from sage.crypto.sboxes import PRESENT
ciph = SBoxCipher(input_length=16, output_length=8, name="ciph")
sb = SBox_CVL(PRESENT)
modadd = ModAdd_CVL(word_length=8)

# Causes an error:
# TypeError: The passed sub_cipher has type
# <class 'civerly.component.ModAdd_CVL'>
# and is not allowed in SBoxCiphers.
ciph.add_subcipher(modadd, [(ciph.IN, (i, i)) for i in range(16)])

# In constrast, this works:
ciph.add_subcipher(sb, [(ciph.IN, (i, i)) for i in range(4)])
```

We also differentiate between ciphers that come with a word structure and those that do not by introducing the `WordBasedCipher` class. Here, all components that are added are required to have input and output sizes that are multiples of the word size specified upon initialization of the respective `WordBasedCipher` object. This forms the base class of three different subclasses which, next to enforcing a fixed word size, specify the non-linear component they support. Objects from the `WordSBoxCipher` class allow S-boxes, while `AddRX` and `AndRX` instances only allow modular addition and logical AND, respectively. The two latter types of non-linear components are not supported in the MILP part of CiVerLy as there exist much more efficient SAT modeling techniques in the literature. Therefore, the corresponding classes are solely meant to be used in the context of SAT modeling. However using the `WordSBoxCipher` class enables the additional features regarding word-wise MILP modeling, while also supporting the bitwise methods introduced by the `SBoxCipher` parent class.

Finally, as the AES influenced the design of many symmetric primitives, we add a special class for such AESlike ciphers, which are required to operate on a rectangular state matrix while inheriting the properties of `WordSBoxCipher` objects. Furthermore, linear layers implemented as `LinearLayer_CVL` objects are required to operate on columns, i.e. their input and output size must be equal to the ciphers word size times the number of rows in the ciphers state. However this does not hold for `PermuteLayer_CVL`, as these might resemble `ShiftRows`-like components which act on the entire state. Furthermore, it is important to consider the inconsistent indexing in AES-like state arrays in the literature, where some ciphers such as `Craft` enumerate the words in a row-wise manner, while other ciphers such as the `AES` uses column-wise indexing. Due to the importance of the `AES` in symmetric cryptography, we decided to employ column-wise indexing. Therefore, those AES-like ciphers that use row-wise indexing need to be transposed before and after applying a similar permutation.

An important note is that generally, a cipher should be implemented as low in the hierarchy as possible as this maximizes the number of available model options. For a concise summary of the class structure in CiVerLy see the table below.

Table 1: Summary of each Cipher subclass together with its supported components and allowed modeling techniques.

Class	Components allowed	Modeling techniques (MILP)	Modeling techniques (SAT)
Cipher	All	-	Bitwise
SBoxCipher	All except AND_CVL, ModAdd_CVL	Bitwise	Bitwise
WordBasedCipher	All with correct wordsize	-	Bitwise
WordSBoxCipher	See WordBasedCipher and SBoxCipher	Bitwise, Wordwise	Bitwise
AndRX	See WordBasedCipher, except SBox_CVL, ModAdd_CVL	-	Bitwise
AddRX	See WordBasedCipher, except SBox_CVL, AND_CVL	-	Bitwise
AESlike	See WordSBoxCipher, only MixColumn-like LinearLayer_CVL	Bitwise, Wordwise	Bitwise

To implement a cipher, we first initiate an empty instance of the corresponding cipher class. Then, step by step, we add the sub-ciphers or components that make up the computational circuit of the cipher. This is implemented with the `add_subcipher` method of the `Cipher` object which takes the component/sub-cipher and the origin of its inputs as parameters. Conceptually, this means that we build a Directed Acyclic Graph (DAG) where each node is either a (sub-)cipher or a fundamental component and the edges describe the dataflow between these. For a full list of the available components, we refer to the CiVerLy documentation. Notice that there is a special node in the cipher that represents its inputs, which is inserted into the DAG upon initialization. The last step of implementing a cipher is to declare its outputs which is done by using the `add_output` method. If not all outputs have been specified, the cipher is not considered to be finished and therefore can not be evaluated nor modeled. Whether this is the case or not is indicated by the `is_valid` attribute, which is only set to `True` if all outputs have been specified. Once we have finished implementing the cipher, we can evaluate it, e.g., to verify test vectors, by simply calling its `eval` method.

2.1.2 Modeling the Cipher

After a cipher has been implemented in CiVerLy, the next step is to model the cipher. While CiVerLy takes care of the modeling, the user has to specify what models CiVerLy shall use. Before we give an overview of the available options and make recommendations for defaults, we briefly explain the high-level idea of the modeling procedure.

How Ciphers Are Modeled

The implementation of a cipher in CiVerLy reassembles a directed acyclic graph. The components in the cipher (and its subciphers) are nodes in the graph and their connections are the edges. CiVerLy uses this structure when modeling a cipher: First, all components are modeled individually. This is possible because the CiVerLy source code contains procedures to model all available components. Notice that depending on the chosen model and component, this process can take some time but usually finishes in a couple of minutes. CiVerLy then connects the individual models into one model for the complete cipher and further adds an objective function or a weight bound for MILP and SAT respectively.

Choosing the Model Options

Below, we give an exemplary code snippet that configures the model options for an CiVerLy analysis.

```
sage: from civerly.model_options import *
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.SAT,
....:     granularity=GRANULARITY.BITWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.EXCLUDE_ODD,
....:     sbbox_modeling=SBOX_MODELING.LOGICAL_COND_ESPRESSO,
....:     solver=SOLVER.CRYPTOMINISAT,
....:     solve_range=(0, 32),
....:     path=Path("Models/Cipher"))
```

While some of the options should be clear already from the example, e.g., `cryptanalysis` tells CiVerLy whether to apply differential or linear cryptanalysis, we list all available options below. Notice that in the source code all this options are gathered in `model_options.py` (hence the first line in the example).

General Options

- **cryptanalysis**

With this mandatory option the user chooses between differential and linear cryptanalysis. That is, `cryptanalysis` must be set to:

- `CRYPTANALYSIS.DIFFERENTIAL` or
- `CRYPTANALYSIS.LINEAR`.

- **optimization**

With this mandatory option the user chooses between MILP and SAT modeling. That is, `optimization` must be set to:

- `OPTIMIZATION.MILP` or
- `OPTIMIZATION.SAT`.

- **granularity**

With this mandatory option the user chooses between a bitwise and a wordwise analysis. For wordwise analysis, which is only implemented for MILP, we do not study the specific differences (or masks) throughout the cipher but only the activity pattern. This usually leads to way faster but less precise models. `granularity` must be set to:

- `GRANULARITY.WORDWISE` or
- `GRANULARITY.BITWISE`.

- **solver**

With this option, the user chooses the MILP or SAT solver that CiVerLy will use. The supported MILP solvers are SCIP, GLPK, and Gurobi. For SAT, CiVerLy supports Cryptominisat and CaDiCaL. The solver, of course, must be installed on the same machine as CiVerLy. Notice that this option can also be set to `None`. CiVerLy then only generates the models and leaves it to the user to solve them. Hence, `solver` must be set to:

- `None` or
- `SOLVER.SCIP` or

- `SOLVER.GLPK` or
- `SOLVER.GUROBI` or
- `SOLVER.CRYPTOMINISAT` or
- `SOLVER.CADICAL`.

- **path**

With this mandatory option the user chooses where CiVerLy will store the generated models and other files. That is, `path` is to be understood as directory or folder here and there is no link to the literal *path* in CiVerLy. Internally, CiVerLy relies on the Python `pathlib` library and hence `path` must be a `pathlib.PosixPath` instance. To get this, simply wrap a string describing the directory where CiVerLy should store files into `Path()`, as in the example above. More information about Python's `pathlib` can be found [online](#).

- **solve_range**

This option sets the weight interval in which CiVerLy will generate SAT models. For SAT this default to (0, 100), i.e., by default CiVerLy trails with weight 0 to 100 which is usually a reasonable range. Hence, you can mostly ignore this option for SAT. Ignore this options for MILP.

- **sat_precision**

For certain S-boxes the SAT modeling in CiVerLy is an approximation. This integer parameter determines the precision, i.e., the number of decimal places that are used for the approximation. This default to 0. Increasing it will have a drastic impact on performance.

Techniques to model Components

For certain components, namely linear layers and S-boxes, CiVerLy supports different modeling techniques. We list the corresponding keys below. Of course, if a cipher does not contain any S-box or no linear layer component, then the corresponding options does not have to be set.

- **linear_layer_modeling**

When set, this must be one of:

- `LINEAR_LAYER_MODELING.BRANCH_NUMBER` or
- `LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE` or
- `LINEAR_LAYER_MODELING.CONVEX_HULL` or
- `LINEAR_LAYER_MODELING.MORE_DUMMIES` or
- `LINEAR_LAYER_MODELING.EXCLUDE_ODD`.

- **sbox_modeling**

When set, this must be one of:

- `SBOX_MODELING.CONVEX_HULL` or
- `SBOX_MODELING.LOGICAL_COND` or
- `SBOX_MODELING.DISTORTED_BALL` or
- `SBOX_MODELING.LOGICAL_COND_ESPRESSO`.

Recommendations

Giving general recommendations is, of course, not easy. As every cipher is different, we must expect cases where the following recommendations are non-optimal. Therefore, we strongly recommend to try different options on small instances of a cipher before modeling real instances.

For AES-like ciphers, i.e., cipher consisting of an S-box layer, a step permuting the cells and a MixColumns step, we recommend to use wordwise MILP modeling with the `GENERALIZED_WORDWISE` linear layer modeling. To keep it simple, for everything else we recommend bitwise SAT modeling with the `MORE_DUMMIES` linear layer modeling and the `LOGICAL_COND_ESPRESSO` modeling for S-boxes.

In terms of solvers, for MILP, we recommend SCIP (unless a Gurobi licence is already available). For SAT, we recommend Cryptominisat.

2.1.3 Solving the Model

After CiVerLy modeled a cipher with SAT, the next step is to solve the model. As depicted in the overview in *The CiVerLy workflow*, this is done using an external SAT solver. Notice that this step is expected to be the most time consuming in the CiVerLy workflow.

To this end, for MILP, CiVerLy supports SCIP, GLPK, and Gurobi. Notice that the first two are open source while Gurobi is a commercial product requiring a license to operate. Although we obtained good results with SCIP, the general consensus in the cryptographic community is that commercial MILP solvers are considerably more efficient than the few open source options. As CiVerLy relies on common standards to store MILPs, such can be used externally as well. Here, we also want to point out that there are entities that solve MILPs as a service, see [NEOS Server](#). However, we want to make it crystal clear that the models generated by CiVerLy might be traced back to (at least some information about) the cipher under scrutiny.

In contrast to MILP, for SAT there are many potent open source solvers available. For this project, we agreed on tightly integrating CryptoMiniSat and CaDiCaL into CiVerLy. As CiVerLy makes use of sage to store the generated SAT models in the standard DIMACS [format](#), other solvers can be used externally as well.

Solving Externally

If the used solver is installed on the same machine as CiVerLy, CiVerLy can directly use it and will do so when the `cipher.analyse` function is being used. Of course, you can use CiVerLy also to only generate the models and then solve them externally, e.g., on a different machine.

MILP

For MILP, it suffices to run the `cipher.model` function to generate a MILP stored in a `.mps` file, a common format supported by MILP solvers. This can then be fed into your solver of choice. As mentioned before, solving the MILP is often expected to take a significant amount of time. However, notice that most MILP solvers will report meaningful intermediate results.

Below, we exemplary depict the output of the three solvers SCIP, Gurobi and GLPK when solving a MILP for 10-round AES. As this is a rather simple problem, SCIP and Gurobi finish nearly instantly, but for more complex problems the solvers will report the current state every few seconds.

Most importantly for us, this state contains the current bounds (both upper and lower) on the objective value, i.e., here the number of active S-boxes. SCIP reports them as `dualbound` and `primalbound`, Gurobi reports them as `Incumbent` and `BestBd` and GLPK simply states both of them with a `>=` between. Notice that the actual values are sometimes reported in scientific notation, i.e., using e.g. `e+01` to indicate a factor of 10.

For bit-wise models, the objective value is the weight, i.e., the negative logarithm with base 2 of the probability, of the best found trail. For word-wise models, the objective value is the number of active S-boxes. Hence, from an attacker's view, the upper bound corresponds to the complexity of the best attack found so far. On the other hand, the lower bound corresponds to a proof that no attack can use a trail with less active S-boxes (or a higher probability resp.) than given by the bound. Therefore, these bounds can provide valuable insights on the security of the cipher under scrutiny (possibly long) *before* the solver terminates.

SCIP

time	node	...	dualbound	primalbound	gap	compl.
p 0.1s	1	...	0.000000e+00	6.400000e+01	Inf	unknown
p 0.1s	1	...	0.000000e+00	5.500000e+01	Inf	unknown
0.1s	1	...	1.000000e+01	5.500000e+01	450.00%	unknown
0.1s	1	...	1.300000e+01	5.500000e+01	323.08%	unknown
0.1s	1	...	1.300000e+01	5.500000e+01	323.08%	unknown
...						
0.7s	1	...	1.794022e+01	5.500000e+01	206.57%	unknown
0.7s	1	...	1.794023e+01	5.500000e+01	206.57%	unknown
time	node	...	dualbound	primalbound	gap	compl.
1.1s	1	...	1.798067e+01	5.500000e+01	205.88%	unknown

Gurobi

Nodes		Current Node			Objective Bounds			Work	
Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
	0	0	10.000000	0	99	100.000000	10.000000	90.0%	0s
H	0	0				80.00000000	10.000000	87.5%	0s
H	0	0				65.00000000	10.000000	84.6%	0s
H	0	0				60.00000000	10.000000	83.3%	0s
	0	0	11.41032	0	164	60.000000	11.41032	81.0%	0s
	0	0	49.30925	0	77	60.000000	49.30925	17.8%	0s
	0	0	52.87272	0	91	60.000000	52.87272	11.9%	0s
	0	0	53.00958	0	97	60.000000	53.00958	11.7%	0s
*	0	0		0		55.00000000	55.000000	0.00%	0s

GLPK

+ 2460:	mip =	not found yet >=	-inf	(1; 0)
+ 7278:	mip =	not found yet >=	1.0000000000e+01	(42; 7)
+ 13488:	mip =	not found yet >=	1.0000000000e+01	(83; 16)
+ 20528:	mip =	not found yet >=	1.3000000000e+01	(126; 26)
...				
+107426:	>>>>>	7.7000000000e+01 >=	2.1000000000e+01	72.7% (872; 101)
+114245:	mip =	7.7000000000e+01 >=	2.2000000000e+01	71.4% (936; 108)
+120664:	mip =	7.7000000000e+01 >=	2.2000000000e+01	71.4% (1013; 111)
+126752:	>>>>>	6.7000000000e+01 >=	2.2000000000e+01	67.2% (1078; 113)
+132989:	mip =	6.7000000000e+01 >=	2.2000000000e+01	67.2% (1109; 167)

(continues on next page)

(continued from previous page)

+140043: mip =	6.700000000e+01	>=	2.200000000e+01	67.2% (1167; 173)
+146966: mip =	6.700000000e+01	>=	2.200000000e+01	67.2% (1233; 178)
+153555: mip =	6.700000000e+01	>=	2.300000000e+01	65.7% (1294; 184)

SAT

As SAT is a decision and not an optimization problem, we need multiple SAT instances to find an optimal trail. To generate those, set the solver to `None` in the model options and then run the `cipher.analyse` function. This will generate model that encode the question whether there exists a trail with weight less or equal W for all W in the solve range, i.e., for $0 \leq W < 100$ per default. You can tackle these SATs then with a SAT solver of you choice. Every time you solve a problem, your bound improves. That is, when you solve the problem for the weight bound W , you get either SAT or UNSAT. If the result is SAT, this means that the optimal trail has weight less or equal W . If the result is UNSAT, this means that the optimal trail has weight greater than W .

2.1.4 Generating the Report

The solution files generated by solvers contain all the necessary information for the analysis of ciphers, namely both the optimal bound as well as the variable assignments responsible for that bound. % Depending on the specific goal of the CiVerLy user, the optimal bound might give sufficient information about the analysed cipher already. In that case, the optimal bound may be read off directly from the solution file, or it may be extracted via the `_process_solution_file` method from `util.py`.

However, often the actual details of this analysis are of interest to the user, in particular what the activity pattern or trail that yields the given bound actually looks like. In this case it is virtually impossible to gain any direct insight about the underlying cryptographic problem from the solution file, as MILP and SAT modeling are both major steps of abstraction. To help with this, CiVerLy supports two ways of visualizing the trail, either by showing the trail in text form or by generating a PDF-report.

First observe that ciphers are represented as recursive DAGs in CiVerLy. In order to introduce some further structure, we construct a grid of all components based on their depth inside the cipher, which is assigned via Depth-First-Search, where the depth assigned to each subcipher is the maximum number of nodes needed to go from `cipher.IN` to it.

In this way, even ciphers with a non-standard structure can be represented in a somewhat readable way, while the approach is kept as generic as possible.

Then, we display the input and output of each component or subcipher in the report as well the input and output of the cipher itself. If the cipher consists of subciphers, which in turn are cipher objects, we must display the propagation of the found trail inside each of the subciphers too. This is done recursively, by building up a tree structure of the subciphers and components, and subsequently using Breadth-First-Search to determine a fitting node ordering.

Note that a priori, the internal structure of each subcipher is ignored, as only the input and output variables are displayed, while the other variables (possibly associated to components inside the subcipher) are treated as dummy variables with no particular meaning regarding the cipher structure. However, when the subcipher itself is modeled, we need to translate all the previously ignored variables correctly, as now they do in fact resemble the subciphers structure. In order to achieve this, CiVerLy generates translation dictionaries during the modeling step, one for each non-trivial subcipher. These dictionaries are stored as json files and will be applied during the report generation.

This process is best illustrated by an example, for which we consider the toy cipher depicted below. It is intentional that this cipher uses several iterations of subciphers, so that this example becomes more clear, as its report generation will require multiple recursion steps.

A possible implementation of it is given below.

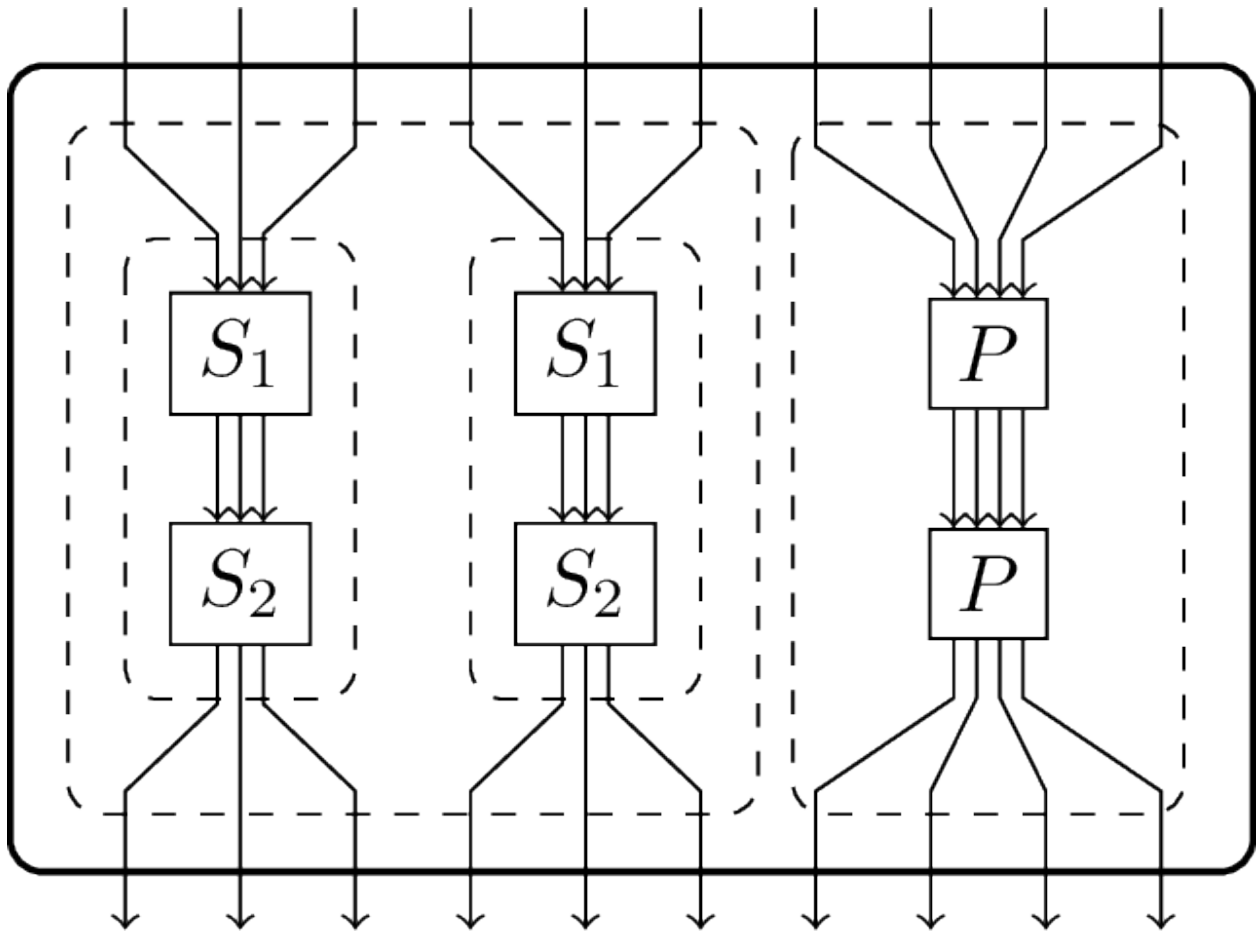


Fig. 3: A toy cipher with several subciphers.

```

sage: from civerly.sboxcipher import SBoxCipher
sage: from civerly.component import PermuteLayer_CVL, SBox_CVL
sage: from sage.crypto.sbox import SBox
sage: cipher = SBoxCipher(10, 10, name="cipher")
sage: subcipher6 = SBoxCipher(6, 6, name="sub-cipher-6-bit")
sage: subcipher3 = SBoxCipher(3, 3, name="sub-sub-cipher-3-bit")
sage: subcipher4 = SBoxCipher(4, 4, name="sub-cipher-4-bit")
sage: sbox1 = SBox_CVL(SBox([0, 7, 3, 5, 1, 2, 6, 4]), name="sbox1")
sage: sbox2 = SBox_CVL(SBox([4, 2, 1, 7, 0, 3, 5, 6]), name="sbox2")
sage: perm = PermuteLayer_CVL([0, 3, 1, 2], name="perm")
sage: node = subcipher3.add_subcipher(
....:     sbox1, [(subcipher3.IN, (i, i)) for i in range(3)]
sage: )
sage: node = subcipher3.add_subcipher(sbox2, [(node, (i, i)) for i in range(3)])
sage: subcipher3.add_output([(node, (i, i)) for i in range(3)])
sage: node1 = subcipher6.add_subcipher(
....:     subcipher3, [(subcipher6.IN, (i, i)) for i in range(3)]
sage: )
sage: node2 = subcipher6.add_subcipher(
....:     subcipher3, [(subcipher6.IN, (i + 3, i)) for i in range(3)]
sage: )
sage: subcipher6.add_output([(node1, (i, i)) for i in range(3)])
sage: subcipher6.add_output([(node2, (i, i + 3)) for i in range(3)])
sage: node = subcipher4.add_subcipher(perm, [(subcipher4.IN, (i, i)) for i in range(4)])
sage: node = subcipher4.add_subcipher(perm, [(node, (i, i)) for i in range(4)])
sage: subcipher4.add_output([(node, (i, i)) for i in range(4)])
sage: node1 = cipher.add_subcipher(
....:     subcipher6, [(cipher.IN, (i, i)) for i in range(6)]
sage: )
sage: node2 = cipher.add_subcipher(
....:     subcipher4, [(cipher.IN, (i + 6, i)) for i in range(4)]
sage: )
sage: cipher.add_output([(node1, (i, i)) for i in range(6)])
sage: cipher.add_output([(node2, (i, i + 6)) for i in range(4)])

```

First, we analyse the toy cipher:

```

sage: from civerly.model_options import *
sage: model_options = MODEL_OPTIONS(
    cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
    optimization=OPTIMIZATION.MILP,
    granularity=GRANULARITY.BITWISE,
    sbox_modeling=SBOX_MODELING.LOGICAL_COND,
    solver=SOLVER.SCHIP,
    path=Path("./Example")
)
cipher.analyse(model_options)

```

As we have thereby also modeled the cipher, we see below that (next to the .mps, .sol and .log files) CiVerLy generated json files containing the corresponding dictionaries. Notice that files named <name>_d.json contain the dictionaries, while files of the form <name>_id.json contain their inverses, which are sometimes necessary when the key of a specific value must be found.

Example

```

— cipher_d.json
— cipher_id.json
— cipher SCIP.log
— cipher.mps
— cipher.sol
— sub-cipher-4-bit_d.json
— sub-cipher-4-bit_id.json
— sub-cipher-6-bit_d.json
— sub-cipher-6-bit_id.json
— sub-sub-cipher-3-bit_d.json
— sub-sub-cipher-3-bit_id.json

```

Let us inspect the content of `cipher_d.json` more closely, to give a better intuition for how to understand the structure of these dictionaries.

```

[
{ # dictionary for cipher.IN (cipher.complist[0])
  "X0[0]": "OUT[0]", "X0[1]": "IN[0]", "X0[2]": "OUT[1]",
  "X0[3]": "IN[1]", "X0[4]": "OUT[2]", "X0[5]": "IN[2]",
  "X0[6]": "OUT[3]", "X0[7]": "IN[3]", "X0[8]": "OUT[4]",
  "X0[9]": "IN[4]", "X0[10]": "OUT[5]", "X0[11]": "IN[5]",
  "X0[12]": "OUT[6]", "X0[13]": "IN[6]", "X0[14]": "OUT[7]",
  "X0[15]": "IN[7]", "X0[16]": "OUT[8]", "X0[17]": "IN[8]",
  "X0[18]": "OUT[9]", "X0[19]": "IN[9]"
},
# dictionary for subcipher6 (cipher.complist[1])
{"X1[0]": "X0[1]", "X1[1]": "X0[0]", ..., "X1[61]": "X3[3]", ...},
# dictionary for cipher.OUT (cipher.complist[2])
{"X2[0]": "OUT[0]", "X2[1]": "IN[0]", ...},
# dictionary for subcipher4 (cipher.complist[3])
{"X3[0]": "X0[1]", "X3[1]": "X0[0]", ...}
]

```

From this (and checking `cipher.complist` for the DFS ordering of the nodes) we can deduce the variable assignments, e.g. we see that `X0[0]` resembles the 0th output bit of the `cipher.IN` component, or that `X2[1]` corresponds to the 0th input bit of `cipher.OUT`, etc. As mentioned above, this works recursively too. Consider the variable `X1[61]` on the level of `cipher_d`. Inside `cipher.complist[1] = sub-cipher-6-bit`, this translates to `X3[3]`. We continue this process by investigating the file `sub-cipher-6-bit_d.json`, which contains

```
[..., {..., "X3[3]": "X0[2]", ...}, ...]
```

As `cipher.complist[1].complist[3] = sub-sub-cipher-3-bit` (the second one), we continue by analysing `sub-sub-cipher-3-bit_d.json` and see

```
[..., {..., "X0[2]": "OUT[1]", ...}, ...]
```

We may finally check that `cipher.complist[1].complist[3].complist[0] = sub-sub-cipher-3-bit.IN` to conclude our search and learn that the variable `X1[61]` on the level of `cipher` translates to the first output node of the `IN` node inside the second `sub-sub-cipher-3-bit` subcipher inside the `sub-cipher-6-bit` subcipher.

Notice how we recursively iterated through a tree-like structure, where the root node is `cipher` and the children of each node are the subciphers of the corresponding cipher. For our example, the tree structure is depicted below.

Note that this tree (intentionally) contains redundant information, as the activity pattern (i.e. difference or linear mask,

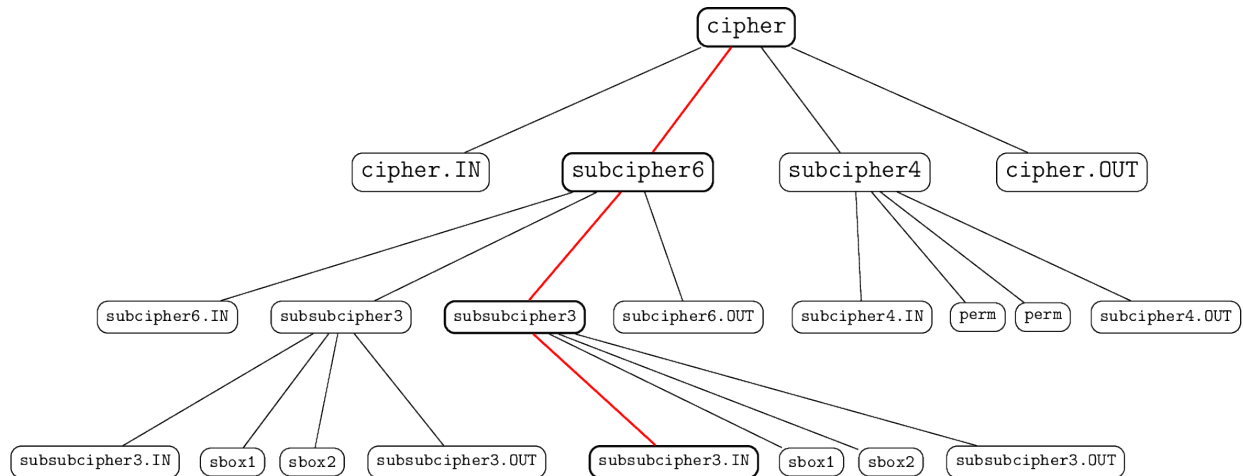


Fig. 4: The tree structure arising from the toy cipher. The node names are shortened for enhanced readability. Red: The traversed path through each dictionary, as explained above.

respectively) in any subcipher is the same as in its respective .IN node. Furthermore, its .OUT node must contain the same activity pattern as the .IN node of the subsequent subcipher. For our example, this means that `cipher` and `cipher.IN` share the same activity pattern, just like `subcipher6.OUT`, `subcipher4` and `subcipher4.IN` share the same value as well.

Text

Now, in regards to the tree structure explained above, calling the method `get_trail` simply returns the nodes in the order of a Depth-First-Search traversal, with whitespace indicating the depth of the current node inside the tree.

```
sage: cipher.get_trail(model_options)
-> sub-cipher-6-bit : 0 -> 0
  -> sub-sub-cipher-3-bit : 0 -> 0
    -> sbox1 : 0 -> 0
    -> sbox2 : 0 -> 0
  -> sub-sub-cipher-3-bit : 0 -> 0
    -> sbox1 : 0 -> 0
    -> sbox2 : 0 -> 0
-> sub-cipher-4-bit : 3c -> 3c
  -> perm : f -> f
  -> perm : f -> f
```

Note that the input into `sub-cipher-4-bit` is translated from `3c` to `f` due to the implicit padding of the report to full nibbles. Therefore, the 10-bit input x is represented as 12-bit $x||00$.

PDF

The tree structure is also utilized to generate the PDF report, however by iterating in a Breadth-First order. Concretely, CiVerLy generates a LaTeX document, of which the first section contains a TikZ-picture of how the trail propagates on the root level. In our example, the components `cipher.IN`, `subcipher6`, `subcipher4` and `cipher.OUT` are displayed in section one of the PDF report. Note that here, the ordering of subciphers is based on DFS, however using the grid-like structure created during the report generation. That is, the depth assigned to each subcipher is the maximum number of nodes needed to go from `cipher.IN` to it, and not the number of nested subciphers it is contained in.

As such a high level description of the trail is often not sufficient, we now iterate this recursively for each subcipher and their respective subciphers etc., as done for the text-based report. Therefore, the second section displays the propagation through `sub-cipher-6-bit`, then for `sub-sub-cipher-3-bit`, etc.

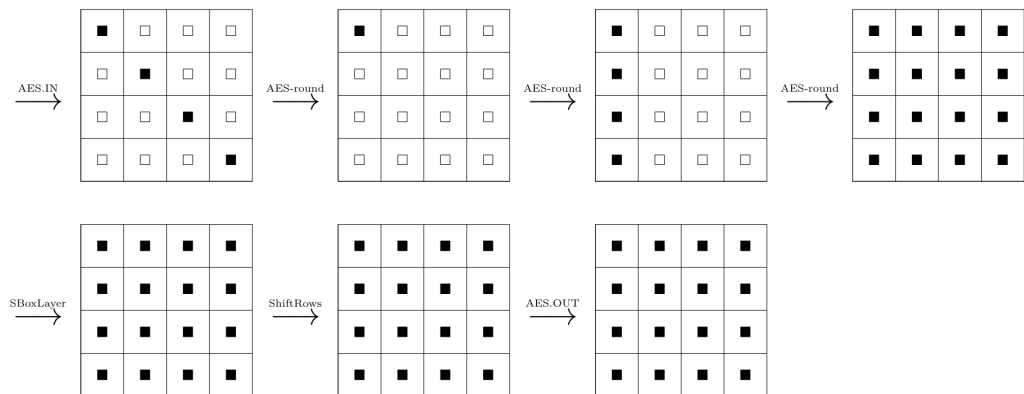
Since the PDF report allows for more customization than the text-based report, we differentiated between the following display options, which are derived from the cipher type as well as the modeling options:

Wordwise Modeling of AESlike Ciphers

The solutions of wordwise models contain assignments to whether a word is active or not, without specifying the concrete difference value (or linear mask). Therefore, there are only two options for each word. When a word is inactive, it will be marked with white square, while being marked with a black square if it is active.

Furthermore, CiVerLy also respects that AESlike ciphers operate on rectangular states and aligns the state accordingly. An example of such a report is the one generated from wordwise modeling of the AES, as shown below.

1 AES



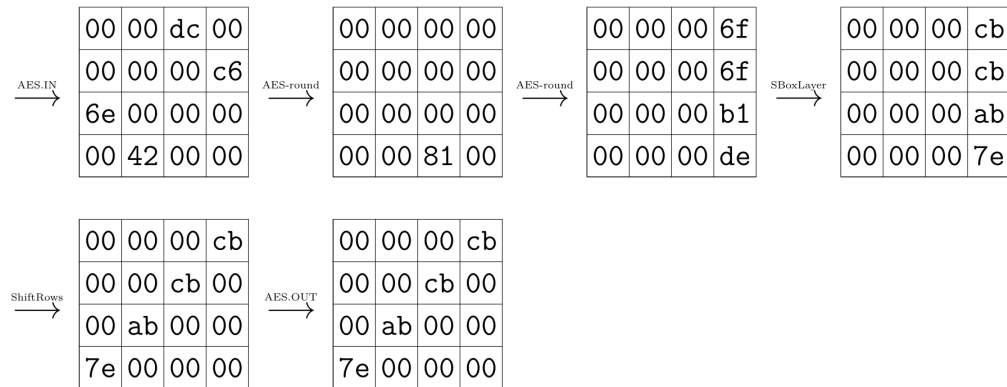
Bitwise Modeling of AESlike Ciphers

For bitwise models, the assignments of each word now contain any values of bitlength `wordsize`. If no `wordsize` was specified, it is defaulted to 4, in order to display nibbles.

To display these, the variables representing each bit are stored into a similar list as for the wordwise modeling. Then, these bits are combined into groups of the correct `wordsize` to display the words.

Again, for AESlike ciphers the rectangular state is displayed accordingly. An example of such a report is the one generated from bitwise modeling of the AES, as shown below.

1 AES



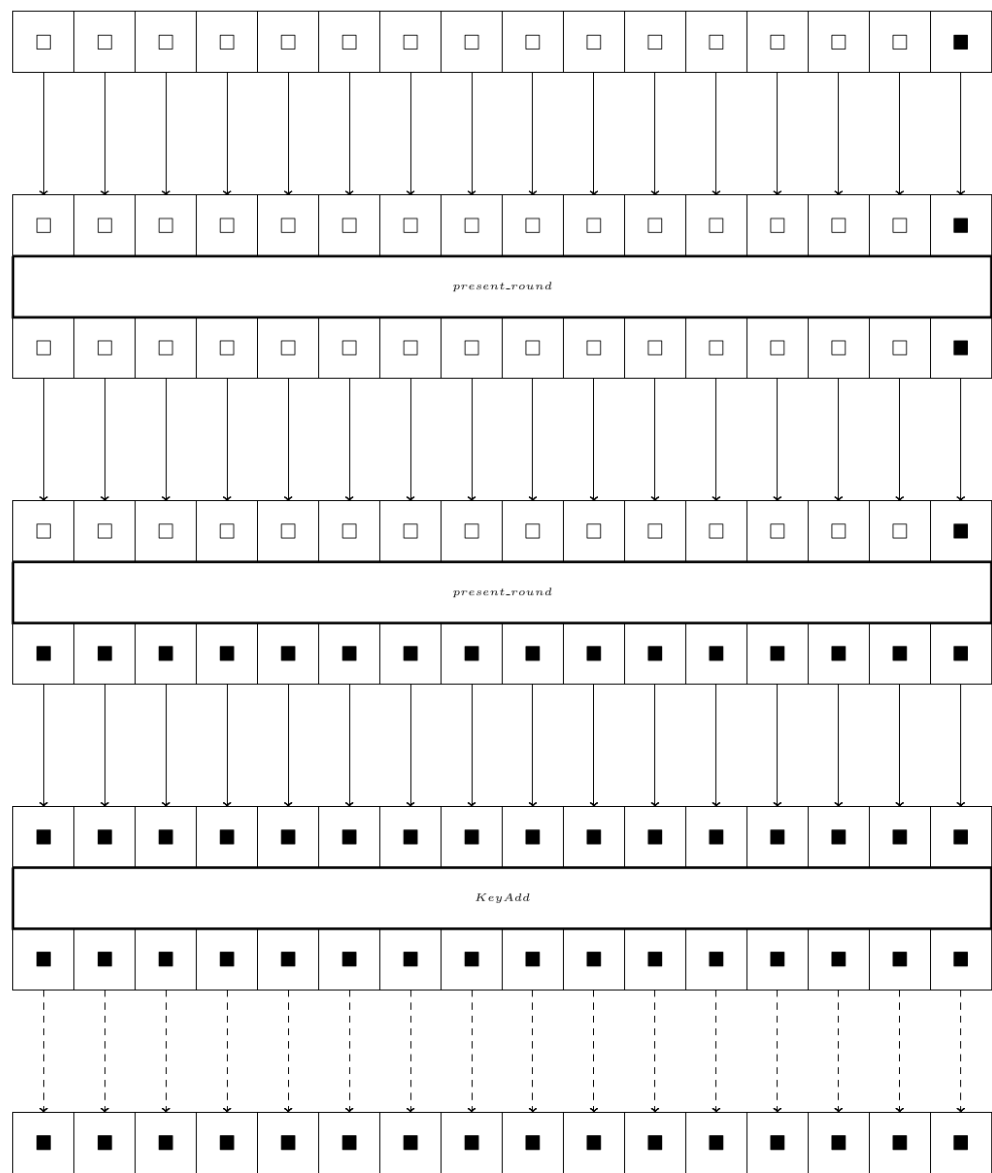
Wordwise Modeling of general Ciphers

Analogously, a report resulting from wordwise modeling of ciphers that are not AESlike (but rather WordSBoxCipher objects, as these are the only ciphers without a rectangular state which support wordwise modeling) looks as follows:

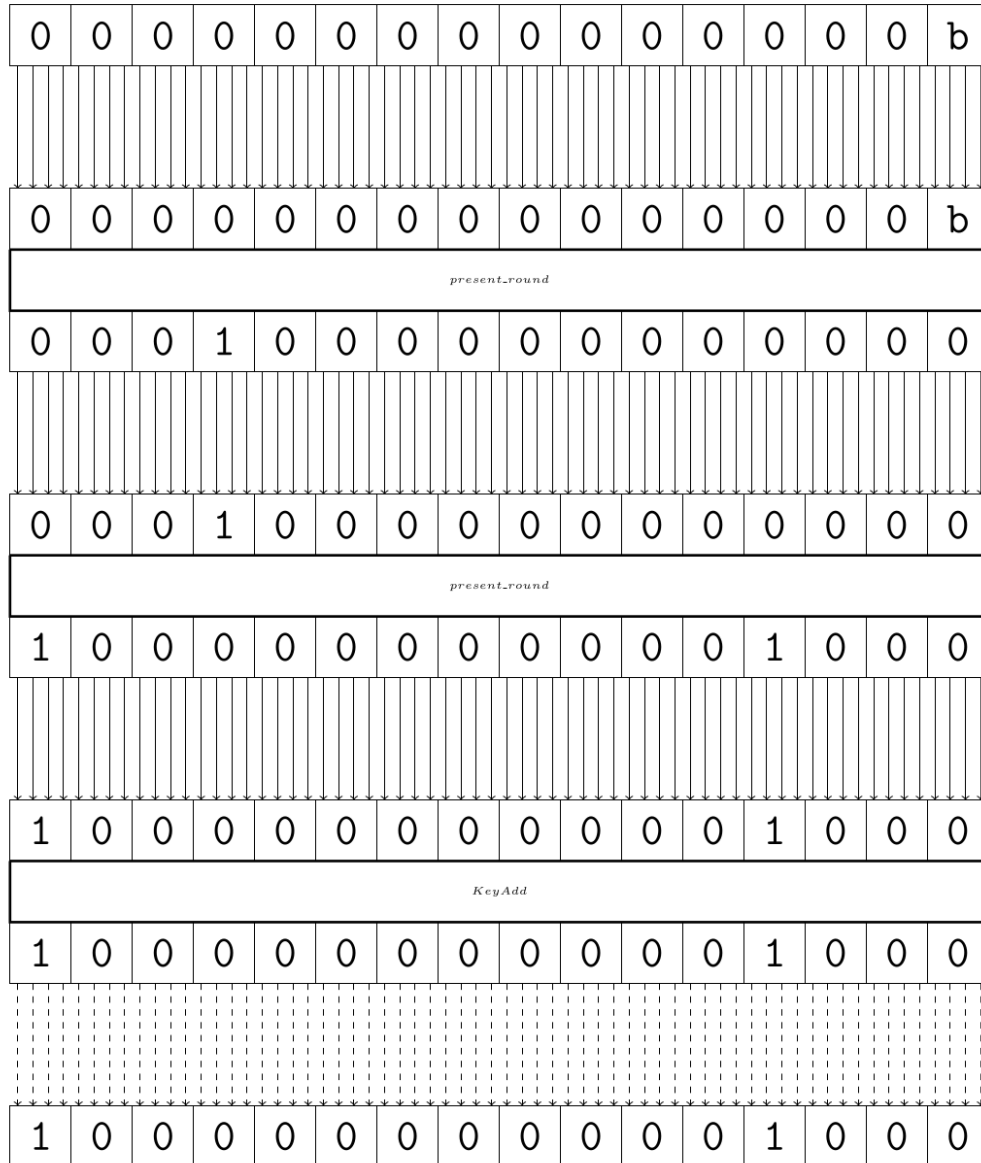
Bitwise Modeling of general Ciphers

Finally, the report from bitwise modeling, which is the one generated from most models, might look like the bitwise PRESENT report as depicted below.

1 PRESENT



1 PRESENT



SOURCE CODE DOCUMENTATION

Here, you find the documentation of the source code of CiVerLy. This sites are automatically generated from the docstrings in the Python files.

3.1 Model Options

Collection of all the available options for modeling.

```
class civerly.model_options.CRYPTANALYSIS(value, names=<not given>, *values, module=None,
                                         qualname=None, type=None, start=1, boundary=None)
```

Bases: Enum

CiVerLy can do differential or linear cryptanalysis.

DIFFERENTIAL = 1

LINEAR = 2

```
class civerly.model_options.OPTIMIZATION(value, names=<not given>, *values, module=None,
                                         qualname=None, type=None, start=1, boundary=None)
```

Bases: Enum

CiVerLy can either generate MILP or SAT models. MILP models can only be generated for objects of SBoxCipher and its subclasses, while SAT models can be generated for objects of any Cipher subclass.

MILP = 1

SAT = 2

```
class civerly.model_options.GRANULARITY(value, names=<not given>, *values, module=None,
                                         qualname=None, type=None, start=1, boundary=None)
```

Bases: Enum

CiVerLy can generate models either on a wordwise or a bitwise level. While wordwise models, which require a WordSBoxCipher or an AESlike cipher, are faster to solve, they are less accurate.

WORDWISE = 1

BITWISE = 2

```
class civerly.model_options.LINEAR_LAYER_MODELING(value, names=<not given>, *values,
                                                    module=None, qualname=None, type=None,
                                                    start=1, boundary=None)
```

Bases: Enum

CiVerLy supports four different techniques to model a linear layer, two for wordwise and two for bitwise modeling.

Wordwise:

- Modeling based on the branch number: make use of the fact that the number of active input and active output words is either zero or at least the branch number
- Generalized wordwise modeling: compute all possible activity patterns for the linear layer and model them using the standard convex hull technique

Bitwise:

- Convex Hull Technique: model each XOR and then compute the convex hull
- Technique with more dummies: Again, model each XOR, but introduce dummy variables to decrease the number of constraints

Note: If your implementation does not contain any `LinearLayer_CVL` component, you can set this to `None`.

BRANCH_NUMBER = 1

GENERALIZED_WORDWISE = 2

CONVEX_HULL = 3

MORE_DUMMIES = 4

EXCLUDE_ODD = 5

```
class civerly.model_options.SBOX_MODELING(value, names=<not given>, *values, module=None,
                                          qualname=None, type=None, start=1, boundary=None)
```

Bases: Enum

CiVerLy supports different techniques for modeling SBoxes.

- Yu Sasaki and Yosuke Todo: New Algorithm for Modeling S-box in {MILP} Based Differential and Division Trail Search (https://doi.org/10.1007/978-3-319-69284-5_11)
- Logical conditioning (<https://eprint.iacr.org/2021/213.pdf>, section 2.3.2) without espresso-reduction
- Christina Boura and Daniel Coggia: Efficient MILP Modelings for Sboxes and Linear Layers of SPN ciphers (<https://doi.org/10.13154/tosc.v2020.i3.327-361>)
- Logical conditioning (<https://eprint.iacr.org/2021/213.pdf>, section 2.3.2) with espresso-reduction (see <https://github.com/classabbyamp/espresso-logic>)

Note: If your implementation does not contain any `SBox_CVL` component, you can set this to `None`.

CONVEX_HULL = 1

LOGICAL_COND = 2

DISTORTED_BALL = 3

LOGICAL_COND_ESPRESSO = 4


```
class civerly.model_options.SOLVER(value, names=<not given>, *values, module=None, qualname=None,
                                     type=None, start=1, boundary=None)
```

Bases: Enum

The solver to be (automatically) used by CiVerLy. Of course, CiVerLy does not implement any solver but simply calls the corresponding solver.

Supported MILP solvers:

- SCIP: Open Source and reasonable performance.
- GLPK: Open Source but only weak performance.
- Gurobi: Commercial solver (license needed). Best performance.

Supported SAT solvers:

- CryptoMiniSat: Open Source solver.
- CaDiCal: Open Source solver.

Note: If you are going to solve all models by yourself (e.g. on a different machine), you can set this to `None`.

Warning: Pick a solver only if it is installed on the same machine you are running CiVerLy on.

SCIP = 1

GLPK = 2

GUROBI = 3

CRYPTOMINISAT = 4

CADICAL = 5

```
class civerly.model_options.MODEL_OPTIONS(cryptanalysis: CRYPTANALYSIS, optimization:
                                             OPTIMIZATION, granularity: GRANULARITY,
                                             linear_layer_modeling: LINEAR_LAYER_MODELING =
                                             None, sbbox_modeling: SBOX_MODELING = None, solver:
                                             SOLVER = None, solve_range: tuple = None, sat_precision:
                                             int = 0, path: Path = None, write_to_file: bool = True)
```

Bases: object

A dataclass to collect the selected options.

INPUT:

- cryptanalysis – see `civerly.model_options.CRYPTANALYSIS`
- optimization – see `civerly.model_options.GRANULARITY`
- linear_layer_modeling – see `civerly.model_options.LINEAR_LAYER_MODELING`
- sbbox_modeling – see `civerly.model_options.SBOX_MODELING`
- solver – see `civerly.model_options.SOLVER`
- solve_range – tuple; The range of weights for which CiVerLy should generate models and solve them
- sat_precision – int; The number of decimal places which is used to find the optimal SAT-bound

- `path` – Path; directory where models will be written to
- `write_to_file` – bool; decides if files are written or not

Below we show how to use a valid configuration of the model options.

EXAMPLES:

```
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.BITWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.CONVEX_HULL,
....:     sbbox_modeling=SBOX_MODELING.CONVEX_HULL,
....:     solver=SOLVER.SCHIP,
....:     path=Path("./CiVerLy-Models/"))
sage: model_options
MODEL_OPTIONS:
-> cryptanalysis : DIFFERENTIAL
-> optimization : MILP
-> granularity : BITWISE
-> linear_layer_modeling : CONVEX_HULL
-> sbbox_modeling : CONVEX_HULL
-> solver : SCIP
-> solve_range : None
-> sat_precision : 0
-> path : CiVerLy-Models
```

cryptanalysis: `CRYPTANALYSIS`

solve_range: `tuple = None`

sat_precision: `int = 0`

exception `civerly.model_options.NoSolverWarning`

Bases: `Warning`

Warning which will be thrown whenever `analyse()` is called with `model_options.solver = None`.

EXAMPLES:

```
sage: from civerly.model_options import *
sage: from civerly.util import suppress_output
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: aes = AES_CVL(6)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,
....:     solver=None,
....:     path=Path("./DOCTEST-ModelOptions/"))
sage: with suppress_output():
....:     aes.analyse(model_options)
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
NoSolverWarning: No solver has been selected.
CiVerLy will return without solving.
```

```
sage: import shutil
sage: shutil.rmtree("DOCTEST-ModelOptions")
```

```
exception civerly.model_options.InvalidModelOptionException(model_option,
                                                             model_option_type=None,
                                                             message=None)
```

Bases: Exception

Exception which will be thrown whenever an invalid model option is given by the user.

3.2 Component

3.2.1 Generic Component

```
class civerly.component.Component(input_length, output_length, name=None)
```

Bases: ABC

Implement the basic structure of all components.

Warning: This component can and **should not be instantiated**. It is rather a superclass of each of the proper components such as SBoxes, XOR, etc.

EXAMPLES:

```
sage: from civerly.component import Component
sage: try: comp = Component(16, 16)
..... except TypeError: print("Did not work")
Did not work
```

For future development

The Component class dictates what methods any subclass should implement. In order for a class to be a functional Component subclass, it needs implement the following (besides inheriting from Component):

- `__init__()` – The init function must specify (implicitly or by taking parameters) the in- and output length of the component.
- `eval()` – Any Component should be possible to evaluate, in order to test the correctness of the implementation with test vectors. `eval()` should accept a bit array as input and should return a bit array on the output as well.
- `__repr__()` – Not necessary to implement, but convenient. When printing the component, a meaningful string should be returned, such as “LL(16 -> 32)” for a linear layer going from a 16-bit input to a 32-bit output.
- `_model_milp()` – Each component must specify how to be modeled with MILP (if at all).
- `_model_sat()` – Each component must specify how to be modeled with SAT (if at all).

```
__init__(input_length, output_length, name=None)
```

Initialize abstract Component class.

INPUT:

- `input_length` – integer; Represents the number of bits going into the component
- `output_length` – integer; Represents the number of bits going out of the component
- `name` – string; The component name for easier identification (optional)

```
model(model_options)
```

Model this component.

This method merely relays `model_options` to an appropriate subroutine.

```
_copy_over_dictionaries_recursively(prev, model_options)
```

3.2.2 Identity Component

```
class civerly.component.I_CVL(input_length, name=None)
```

Bases: [Component](#)

The identity component. It maps any input vector x to x itself, given that x has the correct size (`input_length`).

```
eval(x)
```

Evaluate the output $I_CVL(x) = x$.

3.2.3 Constant Component

```
class civerly.component.C_CVL(output_length, const, name=None)
```

Bases: [Component](#)

Note: In most of the cases, it might be easier to use `ConstXOR_CVL` instead of this.

The constant component. Used e.g. for implementing round constant addition in ciphers. This is a special component, as it doesn't need an input and generates output itself. Furthermore, evaluating this component ignores the input and outputs just the set constant.

INPUT:

- `output_length` – integer; The block size of the component. `input_length` is not a parameter since this component does not need inputs in the Cipher graph.
- `const` – integer; The constant which the component should output upon evaluation.
- `name` – string; The component name for easier identification (optional).

OUTPUT: A constant component initialized with `const`.

3.2.4 RoundKey Component

class `civerly.component.RK_CVL`(*output_length*, *const*, *name=None*)

Bases: `C_CVL`

Note: In most of the cases, it might be easier to use `RoundkeyXOR_CVL` instead of this.

The round-key component. Acts very similarly to `C_CVL`, with the difference that the `const` value is allowed to be changed after instantiation. This allows for using test vectors with non-zero round keys, while treating the key schedules functionality as a blackbox.

INPUT:

- `output_length` – integer; The block size of the component. `input_length` is not a parameter since this component does not need inputs in the Cipher graph.
- `const` – integer; The constant which the component should output upon evaluation.
- `name` – string; The component name for easier identification (optional).

OUTPUT: A round-key component initialized with `const`.

3.2.5 ConstXOR Component

class `civerly.component.ConstXOR_CVL`(*output_length*, *const*, *name=None*)

Bases: `Component`

The `ConstXOR_CVL` component. Used for an easier implementation of round constant addition in ciphers instead of using `C_CVL` and `XOR_CVL` separately.

INPUT:

- `output_length` – integer; The block size of the component.
- `const` – integer; The constant which the component should output upon evaluation.
- `name` – string; The component name for easier identification (optional).

OUTPUT: A `ConstXOR_CVL` component initialized with `const`, which is immutable.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import ConstXOR_CVL
sage: constxor = ConstXOR_CVL(32, 0x11112222)
sage: hex(vec_to_int(constxor(int_to_vec(0xababcdcd, 32))))
'0xbabaefff'
sage: try: constxor.const = 0x1019b214
....: except AttributeError: print("Did not work")
Did not work
```

3.2.6 RoundkeyXOR Component

class civerly.component.RoundkeyXOR_CVL(*output_length, const, name=None*)

Bases: *ConstXOR_CVL*

The RoundkeyXOR_CVL component. Used for an easier implementation of round constant addition in ciphers instead of using C_CVL and XOR_CVL separately. Acts very similarly to the ConstXOR_CVL component, with the difference that the `const` value is allowed to be changed after instantiation. This allows for using test vectors with non-zero round keys, while treating the key schedules functionality as a blackbox.

INPUT:

- `output_length` – integer; The block size of the component.
- `const` – integer; The constant which the component should output upon evaluation.
- `name` – string; The component name for easier identification (optional)

OUTPUT: A RoundkeyXOR_CVL component initialized with `const`.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import RoundkeyXOR_CVL
sage: roundkeyxor = RoundkeyXOR_CVL(32, 0x11112222)
sage: hex(vec_to_int(roundkeyxor(int_to_vec(0xababcdcd, 32))))
'0xbabaefef'
sage: roundkeyxor.const = 0x1019b214
sage: hex(vec_to_int(roundkeyxor(int_to_vec(0xababcdcd, 32))))
'0xbbb27fd9'
```

3.2.7 XOR Component

class civerly.component.XOR_CVL(*word_length, name=None*)

Bases: *Component*

The XOR_CVL component. Accepts the vector $x||y$ and returns $x \oplus y$.

INPUT:

- `word_length` – integer; Determines the word length. The evaluation map will map an input of size $2 \cdot \text{word_length}$ to an output of length `word_length`.
- `name` – string; The component name for easier identification (optional)

OUTPUT: An XOR_CVL instance of length `word_length`.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import XOR_CVL
sage: xor_component = XOR_CVL(32)
sage: hex(vec_to_int(xor_component(int_to_vec(
....: 0x11111111_12121212, 64
....: ))))
'0x3030303'
```

3.2.8 ModAdd Component

class civerly.component.**ModAdd_CVL**(*word_length*, *name=None*)

Bases: *Component*

Modular addition component. Accepts the vector $x||y$ and returns $x \boxplus y$.

INPUT:

- *word_length* – integer; Determines the word length. The evaluation map will map an input of size $2 \cdot \text{word_length}$ to an output of length *word_length*.
- *name* – string; The component name for easier identification (optional).

OUTPUT: An **ModAdd_CVL** instance of length *word_length*.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import ModAdd_CVL
sage: modadd_component = ModAdd_CVL(32)
sage: hex(vec_to_int(modadd_component(
....:   int_to_vec(0x11111111_12121212, 64)
....: )))
'0x23232323'
```

3.2.9 AND Component

class civerly.component.**AND_CVL**(*word_length*, *name=None*)

Bases: *Component*

AND_CVL component. Accepts the vector $x||y$ and returns $x \& y$.

INPUT:

- *word_length* – integer; Determines the word length. The evaluation map will map an input of size $2 \cdot \text{word_length}$ to an output of length *word_length*.
- *name* – string; The component name for easier identification (optional)

OUTPUT: An **AND_CVL** instance of length *word_length*.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import AND_CVL
sage: and_component = AND_CVL(32)
sage: hex(vec_to_int(and_component(
....:   int_to_vec(0x11111111_12121212, 64)
....: )))
'0x10101010'
```

3.2.10 Linear Layer Component

class `civerly.component.LinearLayer_CVL`(*binary_matrix*, *branch_number_differential*=None, *branch_number_linear*=None, *name*=None)

Bases: [Component](#)

The bitwise linear-layer component `LinearLayer_CVL`. The main initialization parameter for the linear layer is the `binary_matrix`. This describes the exact functionality as well as the `input_length` and `output_length`, via its dimensions.

INPUT:

- `binary_matrix` – A SageMath binary matrix; The binary matrix describing the components functionality.
- `branch_number_differential` – integer (optional); Specifies the differential branch number of the `LinearLayer_CVL`. Must be specified to perform branch number based modeling for differential cryptanalysis. The branch number is understood wordwise.
- `branch_number_linear` – integer (optional); Specifies the linear branch number of the `LinearLayer_CVL`. Must be specified to perform branch number based modeling for linear cryptanalysis. The branch number is understood wordwise.
- `name` – string (optional); Specifies the component name.

OUTPUT: A `LinearLayer_CVL` component, which acts as a linear layer transforming bits.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import LinearLayer_CVL
sage: binmatrix = random_matrix(GF(2), 16)
sage: while binmatrix.det() == 0: binmatrix = random_matrix(GF(2), 16)
sage: linearlayer = LinearLayer_CVL(binmatrix)
sage: hex(vec_to_int(linearlayer(int_to_vec(0xabcd, 16)))) # random
'0x37c1'
```

See also:

- More on binary matrices: `sage.matrix.matrix_mod2_dense.Matrix_mod2_dense`

`_milp_bitwise(model_options)`

Bitwise `LinearLayer` modeling, by modeling each XOR. Using one of the following modeling techniques:

- The standard Convex Hull modeling method.
- Method using additional dummy variables.

INPUT:

- `model_options` – see [civerly.model_options.MODEL_OPTIONS](#)

OUTPUT: An object `MixedIntegerLinearProgram`, describing `self` as a MILP.

Warning: The first method needs 2^m steps where m is the highest Hamming weight over the rows of the binary matrix describing the linear layer. That is, this method is infeasible for large and dense linear layers.

property branch_number_differential

Return the branch_number_differential of self.

This branch number is not computed, but simply accepted as an optional argument upon initialization.

property branch_number_linear

Return the branch_number_linear of self.

This branch number is not computed, but simply accepted as an optional argument upon initialization.

3.2.11 Permutation Layer Component

class civerly.component.PermuteLayer_CVL(*perm*, *word_coarseness*=1, *name*=None)

Bases: [LinearLayer_CVL](#)

The PermuteLayer_CVL class, accepting a permutation in form of a python list and converting it into the corresponding binary_matrix, such that most of the functionality (except modeling) is inherited by LinearLayer_CVL.

INPUT:

- *perm* – list; Defines the permutation using the standard notation.
- *word_coarseness* – integer (optional); Specifies the size of the words on which the permutation should act on. For example, if set to 1, a bit permutation is created; if set to 8, a byte permutation is created.
- *name* – string (optional); Specifies the component name.

OUTPUT: A PermuteLayer_CVL component, which acts as a permutation on words of the size word_coarseness.

Note: *perm* starts its indexing at 0 (it is **not** a SageMath permutation).

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import PermuteLayer_CVL
sage: perm = PermuteLayer_CVL([1, 3, 2, 0])
sage: vec_to_int(perm(int_to_vec(0x9, 4)))
12
```

3.2.12 Rotation Layer Component

class civerly.component.RotateLayer_CVL(*input_length*, *r*, *word_coarseness*=1, *name*=None)

Bases: [PermuteLayer_CVL](#)

The RotateLayer_CVL class, accepting a rotation amount and translating it into the corresponding permutation, such that most of the functionality is inherited by PermuteLayer_CVL.

INPUT:

- *input_length* – integer; Determines the size of the rotation window.
- *r* – integer; Determines the rotation amount to the left, can be *input_length* at maximum.
- *word_coarseness* – integer (optional); Specifies the size of the words on which the rotation should act on. For example, if set to 1, bits will be rotated; if set to 8, each byte will be rotated

- `name` – string (optional); Specifies the component name.

OUTPUT: A `RotateLayer_CVL` component, which acts as a rotation of words of the size `word_coarseness`.

Note: This layer rotates to the **left**, i.e. $\lll$

EXAMPLES:

```
sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.component import RotateLayer_CVL
sage: rot = RotateLayer_CVL(16, 4)
sage: vec_to_int(rot(int_to_vec(0x182b, 16)))
33457
sage: vec_to_int(rot(rot(rot(rot(
....:   int_to_vec(0x182b, 16)
....:   ))))) == 0x182b
True
```

3.2.13 SBox Component

class `civerly.component.SBox_CVL(S, name=None)`

Bases: [Component](#)

The `SBox_CVL` class, implementing an SBox.

INPUT:

- `S` – SageMath SBox; Determines the behaviour of the component.
- `name` – string (optional); Specifies the component name.

OUTPUT: An `SBox_CVL` component.

EXAMPLES:

```
sage: from sage.crypto.sboxes import PRESENT
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import SBox_CVL
sage: sb = SBox_CVL(PRESENT)
sage: vec_to_int(sb(int_to_vec(0xd, 4)))
7
```

See also:

- More on SageMath SBoxes: `sage.crypto.sbox.SBox`

3.2.14 AND Component With Rotated Inputs

class `civerly.component.ROT_AND_CVL`(*word_length*, *r*, *name=None*)

Bases: `Component`

This component is specially designed for SIMON-like ciphers. It allows for more precise results when modeling with SAT, since assuming independent inputs when modeling AND_CVL gives highly unrealistic results, especially when in reality the inputs are highly dependent on each other.

EXAMPLES:

```
sage: from civerly.component import ROT_AND_CVL
sage: from civerly.util import int_to_vec, vec_to_int
sage: ra = ROT_AND_CVL(16, 5, name="ra")
sage: vec_to_int(ra(int_to_vec(0x7451, 16)))
0
sage: vec_to_int(ra(int_to_vec(0x1821, 16)))
33
```

property `r`

3.3 Cipher

Cipher is the base class of all ciphers in CiVerLy. A Cipher is either a directed acyclic multigraph of sub-ciphers, or a fundamental component (see `civerly.component.Component`). This allows for a recursive graph structure, simplifying the process of constructing and analysing a cipher. It is possible to evaluate a correctly built cipher in order to test it with test vectors. However, the main functionality of the Cipher object is to model the graph as a MILP or SAT constraint system, which includes multiple modeling techniques. The multigraph of the Cipher object consists of nodes representing the sub-ciphers, as well as edges, indicating on a bitwise level in what way the components are connected.

EXAMPLES:

```
sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.cipher import Cipher
sage: cipher = Cipher(32, 32, name="cipher")
sage: cipher
cipher: 32 -> 32 bits
Sub ciphers:
sage: from civerly.component import I_CVL
sage: identity = I_CVL(32, name="identity")
sage: edges = [(cipher.IN, (i, i)) for i in range(32)]
sage: node = cipher.add_subcipher(identity, edges)
sage: cipher
cipher: 32 -> 32 bits
Sub ciphers:
1: identity: 32 -> 32 bits
sage: cipher.output([(node, (i, i)) for i in range(32)])
sage: hex(vec_to_int(cipher(int_to_vec(0x11119999, 32))))
'0x11119999'
```

exception `civerly.cipher.CipherNotValidException`

Bases: `Exception`

__init__()

Exception which is thrown whenever the cipher is not finished, i.e. not all outputs are connected, while it is required to continue the program flow.

class civerly.cipher.Cipher(*input_length, output_length, name*)

Bases: object

__init__(input_length, output_length, name)

Initialize **self** with the given parameters.

Upon initialization, the cipher is a multigraph containing one node **self.IN**, representing the input of the cipher. Furthermore, it initializes an empty list **outputs** which resembles which bits of which components form the output.

INPUT:

- **input_length** – integer; Represents the block size (number of bits) on the input.
- **output_length** – integer; Represents the block size (number of bits) on the output.
- **name** – string; Used to name and identify the Cipher instance and for naming files that are written to disk.

OUTPUT: The instantiated Cipher with the following attributes:

- **input_length** – integer, public + immutable; Attribute set by the parameter of same name.
- **output_length** – integer, public + immutable; Attribute set by the parameter of same name.
- **name** – string public + immutable; Used to name and identify the Cipher instance and for naming files that are written to disk.
- **is_valid** – bool, private; Used to indicate whether the Cipher has been finished building, as otherwise the output has no incoming nodes (see *is_valid*).
- **IN** – integer, public + immutable; The node label of the input node.
- **nodes** – list, private; List containing all nodes of the graph. Initialized such that it contains the input-node **self.IN**.
- **edges** – list, private; List containing all edges of the graph. Empty when initialized.
- **outputs** – list, private: List containing the node indices that represent the output bits.

An edge is of the form $((a, b), (x, y))$, which corresponds to the fact that the x -th bit of node a is connected with the y -th bit of node b .

Note: An edge is directed, meaning that $((a, b), (x, y)) \neq ((b, a), (y, x))$. Furthermore, an edge must fulfill that $a < b$ in order to avoid cycles in the graph.

property OUT

Return **self.__OUT** whenever **self.OUT** is called.

property is_valid

A boolean variable indicating whether all outputs of **self** have been connected. It is only possible to execute **eval**, **model**, etc. if its value is **True** (see *add_output()* for details on how to achieve this).

add_subcipher(sub_cipher, edges)

Insert **sub_cipher** as a node into the graph, and connects the incoming nodes with it (given by **edges**). The output of **sub_cipher** is not connected any further.

INPUT:

- `sub_cipher` – Cipher; The sub-cipher to be added.
- `edges` – list; The edges connecting the previous nodes with `sub_cipher`.

OUTPUT:

- The node index of the newly added `sub_cipher`.

Each entry in the `edges` list is of the form $(a, (x, y))$, where a represents the index of the node which will be connected to the inputs of `sub_cipher`. More precisely, the x -th bit of a will be connected to the y -th bit of `sub_cipher`.

EXAMPLES:

```
sage: from sage.crypto.sbox import SBox
sage: from civerly.cipher import Cipher
sage: from civerly.component import SBox_CVL
sage: cipher = Cipher(9, 9, "Cipher")
sage: sb = SBox_CVL(SBox([0, 6, 1, 4, 2, 3, 5, 7]))
sage: sb2 = SBox_CVL(SBox([5, 6, 7, 1, 2, 0, 4, 3]))
sage: edges = [(cipher.IN, (i, i)) for i in range(3)]
sage: node = cipher.add_subcipher(sb, edges)
sage: edges = [(cipher.IN, (i + 3, i)) for i in range(3)]
sage: node = cipher.add_subcipher(sb, edges)
sage: edges = [(cipher.IN, (i + 6, i)) for i in range(3)]
sage: node = cipher.add_subcipher(sb2, edges)
sage: cipher
Cipher: 9 -> 9 bits
Sub ciphers:
1: Unnamed Component: 3 -> 3 bits
2: Unnamed Component: 3 -> 3 bits
3: Unnamed Component: 3 -> 3 bits
```

`__call__(input_value)`

See [eval\(\)](#).

`eval(plaintext)`

Evaluate `self` on input `plaintext`.

Evaluating a cipher is only possible after all outputs have been connected by [add_output\(\)](#). If not, an error will be thrown.

INPUT:

- `plaintext` – binary vector; The input to be evaluated. Must have the appropriate dimension (`input_length`) in order to succeed.

OUTPUT:

- The evaluation of `self` on `input_value` in form of a binary vector.

See also:

- Converting integers to binary vectors and vice versa: [civerly.util.vec_to_int\(\)](#) and [civerly.util.int_to_vec\(\)](#)
- More on binary vectors: `sage.modules.vector_mod2_dense.Vector_mod2_dense`

EXAMPLES:

```

sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.cipher import Cipher
sage: from civerly.component import SBox_CVL
sage: from sage.crypto.sbox import SBox as SBox_sage
sage: cipher = Cipher(9, 9, name="Cipher")
sage: sb = SBox_CVL(SBox_sage([0, 6, 1, 4, 2, 3, 5, 7]))
sage: edges = [(cipher.IN, (i, i)) for i in range(3)]
sage: node_s0 = cipher.add_subcipher(sb, edges)
sage: edges = [(cipher.IN, (i + 3, i)) for i in range(3)]
sage: node_s1 = cipher.add_subcipher(sb, edges)
sage: edges = [(cipher.IN, (i + 6, i)) for i in range(3)]
sage: node_s2 = cipher.add_subcipher(sb, edges)

```

Now, calling `cipher` results in an exception:

```

sage: vec_to_int(cipher(int_to_vec(0x52, 9)))
Traceback (most recent call last):
...
civerly.cipher.CipherNotValidException:
The cipher is not finished yet, as not all outputs are connected.
Use '.add_output()' to be able to call.

```

In order to be able call, connect the output:

```

sage: cipher.add_output([(node_s0, (i, i)) for i in range(3)])
sage: cipher.add_output([(node_s1, (i, i + 3)) for i in range(3)])
sage: cipher.add_output([(node_s2, (i, i + 6)) for i in range(3)])
sage: hex(vec_to_int(cipher(int_to_vec(0x52, 9))))
'0x189'

```

`add_output(edges)`

Connect the `x`-th bit of node `a` in the graph to the `y` output bit.

INPUT:

- `edges` – list of tuples; The edges that will be connected, in form of a list `[(a, (x, y))]`.

Each entry of `edges` has the following entries:

- `a` – integer; The index of the node that will be connected.
- `x` – integer; The bit position of node `a` which will be connected.
- `y` – integer; The bit position of the output which will be connected.

OUTPUT: None. The `y`-th output bit is now connected.

EXAMPLES:

```

sage: from civerly.cipher import Cipher
sage: from civerly.component import SBox_CVL
sage: from sage.crypto.sbox import SBox
sage: cipher = Cipher(9, 9, "Cipher")
sage: sb = SBox_CVL(SBox([0, 6, 1, 4, 2, 3, 5, 7]))
sage: edges = [(cipher.IN, (i, i)) for i in range(3)]
sage: node_s0 = cipher.add_subcipher(sb, edges)

```

(continues on next page)

(continued from previous page)

```

sage: edges = [(cipher.IN, (i + 3, i)) for i in range(3)]
sage: node_s1 = cipher.add_subcipher(sb, edges)
sage: edges = [(cipher.IN, (i + 6, i)) for i in range(3)]
sage: node_s2 = cipher.add_subcipher(sb, edges)
sage: cipher.is_valid
False
sage: cipher.add_output([(node_s0, (i, i)) for i in range(3)])
sage: cipher.is_valid
False
sage: cipher.add_output([(node_s1, (i, i + 3)) for i in range(3)])
sage: cipher.is_valid
False
sage: cipher.add_output([(node_s2, (i, i + 6)) for i in range(3)])
sage: cipher.is_valid
True

```

Note: Not calling this function correctly (or at all) leads to an error upon evaluation.

model(*model_options*)

Generate the model for `self` according to the given `model_options`. Calls one of the two modeling methods `self._model_milp()` or `self._model_sat()`, depending on the chosen optimization mode in `model_options`.

INPUT:

- `model_options` – see [civerly.model_options.MODEL_OPTIONS](#)

OUTPUT:

- the generated model

analyse(*model_options*)

Analyse this cipher. That is, generate the model, solve it and return the result.

INPUT:

- `model_options` – see [civerly.model_options.MODEL_OPTIONS](#)

Warning: Requires the specified solver to be installed.

generate_report(*model_options*)

Generates a `.tex` file that contains the state matrices of active words after each component, and compiles it to a PDF.

INPUT:

- `model_options` – see [civerly.model_options.MODEL_OPTIONS](#)

OUTPUT: None, but writes a PDF file.

get_trail(*model_options*)

After solving a MILP or SAT, it will convert the solution values to a trail-like output, in form of a list. This is very similar to `self.generate_report`, however the content is in form of a list rather than a pdf file.

3.4 SBoxCipher

The SBoxCipher class, a subclass of Cipher allowing only specific operations.

The SBoxCipher class inherits most of its functionality from Cipher, with the restriction that only sub_ciphers of the type SBoxCipher, SBox_CVL, LinearLayer_CVL, XOR_CVL, RK_CVL, C_CVL are allowed (or its subclasses).

The purpose of this subclass is to allow for MILP modeling, which is not supported for ciphers containing different non-linear components than SBox_CVL, such as AddRX, AndRX.

class `civerly.sboxcipher.SBoxCipher(input_length, output_length, name)`

Bases: [Cipher](#)

`__init__(input_length, output_length, name)`

See also:

- [civerly.cipher.Cipher.__init__\(\)](#) for the initialization details.

`_model_milp(model_options, _first_iter=False)`

Generate the model for self according to the given model_options.

INPUT:

- model_options – see [civerly.model_options.MODEL_OPTIONS](#)

We first generate a master_milp, recursively iterate over self.complist (and its subciphers) and collect the sub-milps in order to relabel and connect them correctly to one big milp. To avoid remodeling the same components multiple times, a caching-mechanism is implemented which checks if the currently modeled component was modeled before. If yes, the milp will be copied over instead of being generated from scratch again.

`_finish_milp(model_options, milp, _first_iter=False)`

Finish the given MixedIntegerLinearProgram. That is, add a constraint that ensures that the input is active and add the objective function. If specified by model_options, write the model to a file.

INPUT:

- model_options – see [civerly.model_options.MODEL_OPTIONS](#)
- milp – MixedIntegerLinearProgram; the milp to be finished

OUTPUT:

- The generated MILP model, describing the possible differential propagations through self.

Warning: If specified by model_options, this method does not only write the model to file but also implicitly generates .json files in the same location, with similar names (<self.name>_d.json and <self.name>_id.json). These are necessary to translate the abstract .mps file content into a meaningful MILP and is therefore used in generate_report to be able to generate the report correctly.

3.5 WordBasedCipher

The `WordBasedCipher` class, a subclass of `Cipher` allowing only specific operations.

The `WordBasedCipher` class inherits most of its functionality from `Cipher`, with the restriction that there has to be a fixed `wordsize`, i.e. each sub-cipher must have an `input_length` and `output_length` that is a multiple of `wordsize`.

This class is the base for the subclass `WordSBoxCipher`, which supports wordwise MILP-modeling, which would be impossible without a fixed `wordsize`. Another purpose of this class is that `.add_subcipher()` of `WordBasedCipher` and its subclasses accepts **wordwise** edges instead of bitwise edges. However this is just a convenience, and internally, there is no difference in the functionality.

EXAMPLES:

```
sage: from civerly.cipher import Cipher
sage: from civerly.wordbasedcipher import WordBasedCipher
sage: from civerly.component import SBox_CVL
sage: from sage.crypto.sboxes import AES
sage: wbc = WordBasedCipher(8, 16, 16, name="WBC")
sage: s8 = SBox_CVL(AES)
sage: node_s8 = wbc.add_subcipher(s8, [(wbc.IN, (1, 0))])
sage: cipher = Cipher(128, 128, name="Cipher")
sage: node_s8 = cipher.add_subcipher(
....:     s8, [(cipher.IN, (i + 8, i)) for i in range(8)])
sage: cipher == wbc
True
```

```
class civerly.wordbasedcipher.WordBasedCipher(wordsize, input_num_words, output_num_words,
                                              name=None)
```

Bases: `Cipher`

```
__init__(wordsize, input_num_words, output_num_words, name=None)
```

See also:

- `Cipher.__init__` for the initialization details.

property `wordsize`

A `WordBasedCipher` has a new attribute, which will be distributed onto its sub-ciphers (of either type `WordBasedCipher` and `Component`) when added.

```
sage: from civerly.wordbasedcipher import WordBasedCipher
sage: cipher = WordBasedCipher(17, 9, 5, name="wordbasedcipher")
sage: cipher.wordsize
17
```

`add_subcipher(sub_cipher, edges)`

The edges are now reduced. Instead of `wordsize` many edges going from a single node into another node, the convention is now that one edge of size `wordsize` goes into the respective node. Consequently, when inserting a `Component` into a `WordBasedCipher`, it receives a new attribute `self.wordsize`.

EXAMPLES:

```
sage: from civerly.cipher import Cipher
sage: from civerly.wordbasedcipher import WordBasedCipher
sage: from civerly.component import SBox_CVL
sage: from sage.crypto.sboxes import AES as AES_S
```

(continues on next page)

(continued from previous page)

```

sage: wbc = WordBasedCipher(8, 16, 16, name="WordBasedCipher")
sage: s8 = SBox_CVL(AES_S)
sage: node_s8 = wbc.add_subcipher(s8, [(wbc.IN, (1, 0))])
sage: cipher = Cipher(128, 128, name="Cipher")
sage: node_s8 = cipher.add_subcipher(
....:     s8, [(cipher.IN, (i + 8, i)) for i in range(8)])
sage: cipher == wbc
True
sage: wbc.wordsize
8
sage: cipher.wordsize
Traceback (most recent call last):
[...]
AttributeError: 'Cipher' object has no attribute 'wordsize'

```

See also:

- `Cipher.add_subcipher`

3.6 WordSBoxCipher

The `WordSBoxCipher` class. Inherits its main functionalities from `WordBasedCipher` and `SBoxCipher`.

A `WordSBoxCipher` combines both functionalities of its parent classes: Any component should have a `input_length` and `output_length` that is a multiple of `self.wordsize`, and is not allowed to be of type `ModAdd_CVL` nor `AND_CVL`.

The main purpose of this class is to support word-wise MILP modeling. This is possible because next to the fixed word size, it is ensured that the ciphers non-linear part comprises only of `SBox_CVL`, which can be modeled using MILP.

```

class civerly.wordsboxcipher.WordSBoxCipher(wordsize, input_num_words, output_num_words,
                                             name=None)

```

Bases: `WordBasedCipher`, `SBoxCipher`

3.7 AES-like

Implementation of the `AESlike` subclass of `Cipher`.

The `AESlike` class assumes a rectangular state matrix of $n \times m$ words of size w . This state will be transformed by the wordwise application of the same w -bit `SBox` in parallel (called an `SBox-layer`), a wordwise permutation (called `PermuteState`), and a column-wise application of a linear layer (called a `MixColumn` operation), which possibly but not necessarily might be described as an operation over $\mathbb{F}_{2^w}$. Finally, a sub-cipher describing the round key addition is allowed to be inserted into an `AESlike` cipher as well. Due to these strict specifications, it is possible to apply a number of modeling techniques to generate MILP/SAT models, since each of the methods possible for any super-class of `AESlike` is applicable here as well. Examples for ciphers that can be implemented as `AESlike` instances are `AES`, `SKINNY`, `MIDORI` and `CRAFT`.

EXAMPLES:

```

sage: from sage.crypto.sboxes import PRESENT
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.component import SBox_CVL, ModAdd_CVL, LinearLayer_CVL

```

(continues on next page)

(continued from previous page)

```
sage: from civerly.aeslike import AESlike
sage: aeslikecipher = AESlike(4, 5, 3, "Cipher") # 5 rows x 3 columns
sage: modadd = ModAdd_CVL(4)
sage: edges = [(aeslikecipher.IN, (0, 0)), (aeslikecipher.IN, (1, 0))]
sage: try: aeslikecipher.add_subcipher(modadd, edges)
....: except TypeError: print("Did not work")
Did not work
sage: sb = SBox_CVL(PRESENT)
sage: for i in range(15):
....:     edges = [(aeslikecipher.IN, (i, 0))]
....:     node = aeslikecipher.add_subcipher(sb, edges)
....:     aeslikecipher.add_output([(node, (0, i))])
```

class `civerly.aeslike.AESlike`(wordsize, rows, cols, name)

Bases: `WordSBoxCipher`

__init__(wordsize, rows, cols, name) → None

AESlike:

The counting in AESlike is column by column, i.e.

0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

INPUT:

```
- ``num_rows`` -- integer; Specifies the number of rows in the
AES-like state matrix.

- ``num_cols`` -- integer; Specifies the number of columns in the
AES-like state matrix.

- ``wordsize`` -- integer; Specifies the wordsize in number of
bits, i.e. the size of each entry in the state matrix.
```

OUTPUT: An AESlike instance.

See also:

- `Cipher.__init__` for the initialization details.

add_subcipher(sub_cipher, edges)

See also:

- `Cipher.add_subcipher`

property rows

Return the number of rows of self.

property cols

Return the number of columns of self.

3.8 AndRX

AndRX is a subclass of `WordBasedCipher`, in which the non-linear components `ModAdd_CVL` and `SBox_CVL` are not allowed.

The main purpose of this class is to support SAT modeling techniques for SIMON-like ciphers.

EXAMPLES:

```
sage: from civerly.andrx import AndRX
sage: from civerly.component import XOR_CVL, AND_CVL, SBox_CVL
sage: from sage.crypto.sboxes import AES
sage: andrx_cipher = AndRX(32, 2, 2, "AndRXCipher")
sage: edges = [(andrx_cipher.IN, (0, 0)), (andrx_cipher.IN, (1, 1))]
sage: node_xor = andrx_cipher.add_subcipher(XOR_CVL(32, name="xor"), edges)
sage: edges = [(node_xor, (0, 0)), (andrx_cipher.IN, (1, 1))]
sage: node_and = andrx_cipher.add_subcipher(AND_CVL(32, name="and"), edges)
sage: try: andrx_cipher.add_subcipher(SBox_CVL(AES), [(node_and, (0, 0))])
....: except TypeError: print("Did not work")
Did not work
```

```
class civerly.andrx.AndRX(wordsize, input_num_words, output_num_words, name=None)
    Bases: WordBasedCipher
```

3.9 AddRX

AddRX is a subclass of `WordBasedCipher`, in which the non-linear components `AND_CVL` and `SBox_CVL` are not allowed.

The main purpose of this class is to support SAT modeling techniques for various AddRX designs.

EXAMPLES:

```
sage: from sage.crypto.sbox import SBox
sage: from civerly.addrx import AddRX
sage: from civerly.component import ModAdd_CVL, AND_CVL, SBox_CVL
sage: addrx = AddRX(4, 4, 4, "Cipher")
sage: edges = [(addrx.IN, (0, 0)), (addrx.IN, (1, 1))]
sage: try: addrx.add_subcipher(AND_CVL(4), edges)
....: except TypeError: print("Did not work")
Did not work
sage: e = [(addrx.IN, (0, 0))]
sage: try: node = addrx.add_subcipher(SBox_CVL(SBox([0,1,4,3,6,5,2,7])), e)
....: except TypeError: print("Did not work")
Did not work
sage: edges = [(addrx.IN, (i, i)) for i in range(4)]
sage: node = addrx.add_subcipher(ModAdd_CVL(8), edges)
```

```
class civerly.addrx.AddRX(wordsize, input_num_words, output_num_words, name=None)
    Bases: WordBasedCipher
```

3.10 Solvers

Utils for interacting with MILP and SAT solvers.

```
class civerly.solvers.SOLVING_STATUS(value, names=<not given>, *values, module=None,
                                     qualname=None, type=None, start=1, boundary=None)
```

Bases: Enum

Indicate if solving was successful or what went wrong.

SUCCESS = 1

TIMEOUT = 2

ERROR = 3

```
civerly.solvers.solve(input_file_name, output_file_name, solver, time_limit=None, log_file_name=None)
```

Solve the given MILP or SAT instance externally.

INPUT:

- `input_file_name`— path to the file containing the MILP or SAT
- `output_file_name`— path of the file the solution is written to
- `solver` — see [civerly.model_options.SOLVER](#)
- `time_limit`— integer (default None); time limit in seconds
- `log_file_name`— path to the solver's log file. Default based on `output_file_name` and `solver`.

OUTPUT:

- Status, see [civerly.solvers.SOLVING_STATUS](#). Further, upon successful execution either a `.sol` or a `.sat` file is created, depending on `solver`. The file will contain the solution, if one is found.

Note: This method is implemented for convenience in case a solver is installed on the same machine as CiVerLy.

```
civerly.solvers.optimize_sat(cnf_file_name, sat_file_name, model_options, time_limit=None,
                             benchmark=False)
```

Repeatedly solve SAT to determine the lowest possible weight.

Given a SAT with its corresponding `sum_arr`, this method applies a binary search to determine the lowest weight `w` for which this SAT is solvable and solves it.

INPUT:

- `cnf_file_name`— path to the file containing the SAT
- `sat_file_name`— path to the file the solution is written to
- `model_options` — see [civerly.model_options.MODEL_OPTIONS](#). Used to retrieve the `solver`, `solve_range` and `sat_precision`.
- `time_limit`— integer (default None); time limit in seconds
- `benchmark`— (default False); when set to True return details about internal timing. This is used by [civerly.benchmark.benchmark\(\)](#).

OUTPUT:

- None, but upon successful execution a `.sol` file is created, containing the solution (if found).

Note: This method is implemented for convenience in case a solver is installed on the same machine as CiVerLy.

`civerly.solvers._float_or_int(value)`

Cast *value* to float or int if possible.

EXAMPLES:

```
sage: from civerly.solvers import _float_or_int
sage: _float_or_int(42.0)
42
sage: _float_or_int(42.5)
42.5
sage: _float_or_int("42.000")
42
sage: _float_or_int("42.001")
42.001
sage: _float_or_int("3.415037499300e+00")
3.4150374993
```

`civerly.solvers.get_objective_bounds(log_file_name, solver)`

Extract the bounds on the objective value from the log of an MILP solver.

This function shall be used when the MILP solver exceeds the given timeout.

INPUT:

- `log_file_name`— name of the log file
- `solver` — the used solver, see `civerly.model_options.SOLVER`

`civerly.solvers.get_objective_value(sol_file_name, solver)`

Extract the objective value from the specified solution file.

INPUT:

- `sol_file_name`— name of the file containing a solution
- `solver` — the used solver, see `civerly.model_options.SOLVER`

`civerly.solvers._process_solution_file(solution_file_name, solver)`

Internal helper method to process the solution file.

INPUT:

- `solution_file_name`— name of the file containing a solution
- `solver` — the used solver, see `civerly.model_options.SOLVER`

OUTPUT: The processed results and `objective_value`.

`civerly.solvers.__parse_cryptominisat(file_content)`

Parse ‘.sat’ files and recovers the variable assignments as well as the objective value.

Note: CiVerLy augments the standard file format convention for SAT output files. Typically the first line holds the string ‘SAT’ or ‘UNSAT’, while the variable assignments are given in the second line, in form of the **sign** of the corresponding integer. Additionally, CiVerLy writes the objective value into the second line, as an integer, which is non-standard.

`civerly.solvers.__parse_cadical(file_content)`

Parse a solution generated by CaDiCal.

3.11 Utility functions

Util class of CiVerLy

Contains utility functions.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: int_to_vec(0x1234, 16)
(0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0)
sage: int_to_vec(0x1234, 32)
(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0,
0, 0, 1, 1, 0, 1, 0, 0)
sage: type(int_to_vec(0x1234, 32))
<class 'sage.modules.vector_mod2_dense.Vector_mod2_dense'>
sage: vec_to_int(int_to_vec(0x1234,16)) == 0x1234
True
sage: try: int_to_vec(0x1234, 12)
....: except ValueError: print("Did not work")
Did not work
```

`civerly.util.vec_to_int(input_vec)`

Converts a binary vector (see `sage.modules.vector_mod2_dense.Vector_mod2_dense` for details) into the corresponding integer.

INPUT:

- `input_vec` – Binary vector; Represents the input vector.

OUTPUT: The integer represented by `input_vec`.

EXAMPLES:

```
sage: from civerly.util import vec_to_int
sage: hex(vec_to_int(vector(
....:   GF(2),
....:   [1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0]
....: )))
'0x4f51a'
```

`civerly.util.int_to_vec(input_num, size)`

Converts an integer into the corresponding binary vector (see `sage.modules.vector_mod2_dense.Vector_mod2_dense` for details).

INPUT:

- `input_num` – integer; Represents the integer input.
- `size` – integer; The length of the resulting vector

OUTPUT: The binary vector representing `input_num`.

EXAMPLES:

```
sage: from civerly.util import int_to_vec
sage: int_to_vec(0x12340, 24)
(0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0,
1, 0, 0, 0, 0, 0, 0, 0)
sage: try: int_to_vec(0x12340, 16)
..... except ValueError: print("Did not work")
Did not work
```

`civerly.util.translate_sat_clause(VAR, clause)`

Given a clause in form of a tuple and an Iterable VAR, translate the clause into a clause using the VAR entries.

Note: This not only works for SAT clauses, but also to translate SAT clauses into MILP constraints.

EXAMPLE:

```
sage: from civerly.util import translate_sat_clause
sage: VAR = [11, 22, 33, 44, 55]
sage: clause = (1, -2, 3, -4, -5)
sage: translate_sat_clause(VAR, clause)
(11, -22, 33, -44, -55)
sage: from sage.numerical.mip import MixedIntegerLinearProgram
sage: milp = MixedIntegerLinearProgram(
.....:     maximization=False, solver="GLPK"
.....: )
sage: VAR_milp = milp.new_variable(name="VAR", binary=True)
sage: translate_sat_clause(VAR_milp, clause)
(x_0, -1*x_1, x_2, -1*x_3, -1*x_4)
```

`civerly.util.translate_milp_constraint(VAR, constr)`

Given a MILP constraint and an Iterable of MILP variables VAR, translate the constraint into a new constraint using the entries of VAR.

`civerly.util._generate_constraints_sum_leq_int_LS24(sat, sum_arr, num)`

Helper method to implement a sequential counter in SAT, in order to model the constraint $\text{sum}(\text{arr}) \leq \text{num}$ as a SAT-formula. The constraints are given in https://link.springer.com/chapter/10.1007/978-3-031-54776-8_14, section 3.4.

Note: There is a typo in this paper, as the sum that we want to bound should go from $0 \leq i \leq l - 1$ instead of l . Therefore, the last constraint must be $u_{l-1}^- \vee a_{l-2,w-1}$ instead of $\bar{u}_l \vee a_{l-1,w-1}$.

`civerly.util._sat_get_clauses_espresso(posset)`

Invoke Espresso to produce a (close to) minimal set of CNF-clauses that are only fulfilled for elements in posset.

INPUT:

- posset – list of lists/vectors; Contains all possible transitions

OUTPUT: clauses – a close to minimal set of CNF-clauses that describe posset

`civerly.util.reduction_algorithm_ST17(comp, posset, model_options, PROB=None)`

Implements Yosuke Todos and Yu Sasaki Reduction Algorithm (https://link.springer.com/chapter/10.1007/978-3-319-69284-5_11), which models the problem of choosing a minimal subset of MILP-constraints as a MILP itself. Intended to be used internally.

3.12 Benchmark

Benchmark CiVerLy.

`civerly.benchmark.benchmark(CM=None, remove_files=False, only_models=False, solving_time_limit=None)`

Generate benchmarks for the given ciphers and models.

The results are printed as latex code.

INPUT:

- `CM` – list; list of tuples (`ciphers`, `model_options`) where `ciphers` is a list of ciphers.
- `solving_time_limit` – integer (default `None`); time limit (in seconds) for solving an individual model

EXAMPLES:

```
sage: from civerly.benchmark import benchmark
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: path = Path("./DOCTEST-Benchmark-AES-Models/")
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.BRANCH_NUMBER,
....:     solver=SOLVER.SCIP,
....:     path=path)
sage: aes = [AES_CVL(R=r, name=f"{r}r-AES") for r in range(1, 10)]
sage: CM = [(aes, model_options)]
sage: benchmark(CM) # optional - scip
...
```

3.13 Implementations

These are implementations of some well-known ciphers.

3.13.1 ABC

`class civerly.cipher_implementations.abc.ABC_CVL(*args, **kwargs)`

Bases: object

`__init__(R=16, rks=[], name=None)`

The ABC cipher, a 128 bit block size Feistel cipher patented by Apple and first analysed in https://link.springer.com/chapter/10.1007/978-3-031-56232-7_13. The main weakness is the lack of diffusion between each byte of the state, which CiVerLy also recognizes.

3.13.2 AES

Implementation of the AES.

Arguably, the AES is the most important cipher in the world. Hence, we use it to exemplary describe the usage of CiVerLy.

EXAMPLES:

First, we want to determine the number of active S-boxes in 10-round AES. For this we will use a word-wise model based on the the branch number. As AES is already implement in CiVerLy, we can simply import it and instantiate it:

```
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: aes = AES_CVL(R=10)
```

Next, we have to tell CiVerLy the specifics of the modeling technique we want to use. A full list is given in `civerly.model_options.MODEL_OPTIONS` but the code below should be rather straightforward:

```
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.BRANCH_NUMBER,
....:     solver=SOLVER.SCIIP,
....:     path=Path("./DOCTEST-AES-Models/"))
```

Notice that we set `sbox_modeling` to `None`, as we do not have to model the S-box in our wordwise model. Furthermore, the specified path is used for storing the generated models. Next, we simply tell CiVerLy to analyse AES:

```
sage: # optional - scip
sage: aes.analyse(model_options) # doctest: +NORMALIZE_WHITESPACE
2884 variables and 3085 constraints were written to
'DOCTEST-AES-Models/AES.mps'
55
```

Indeed, there are 55 active S-boxes for 10-round AES. We clean up the generated files:

```
sage: import shutil
sage: shutil.rmtree("DOCTEST-AES-Models", ignore_errors=True)
```

Next up, we want to study linear cryptanalysis using the more accurate modeling of MixColumn which requires solving a MILP itself:

```
sage: # optional - scip
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: aes = AES_CVL(R=10)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.LINEAR,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,
....:     solver=SOLVER.SCIIP,
....:     path=Path("./DOCTEST-AES-Models/"))
sage: aes.analyse(model_options) # doctest: +NORMALIZE_WHITESPACE
```

(continues on next page)

(continued from previous page)

```

2848 variables and 2977 constraints were written to
'DOCTEST-AES-Models/AES.mps'
55
sage: shutil.rmtree("DOCTEST-AES-Models", ignore_errors=True)

```

Notice that above we use the analyse function of the aes cipher. While this is rather convenient, it requires the specified solver to be installed on the same machine as CiVerLy. However, (for more complex models) you might want to solve the models on a different, more powerful machine. We adapt the example from above for this case. First, we again define the cipher and the model_options:

```

sage: from civerly.cipher_implementations.aes import AES_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: aes = AES_CVL(R=10)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.LINEAR,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,
....:     path=Path("./DOCTEST-AES-Models/"))

```

As we rely on an external solver, we can simply put it to None in the model_options. Next, we generate the model:

```

sage: aes.model(model_options) # doctest: +NORMALIZE_WHITESPACE
LinearLayer MILP has been written to
DOCTEST-AES-Models/MixColumn51845.mps.
In order to continue the modeling, solve the generated MILP by providing a
solution file with the name DOCTEST-AES-Models/MixColumn51845.sol.

```

As the output states, you have to provide a solution to the MILP for MixColumn to continue the modeling. You would copy the .mps file to a machine with a supported MILP solver, solve it and copy the .sol to the location specified above. As we cannot execute that step here, we simulate it to continue:

```

sage: # optional - scip
sage: from civerly.solvers import solve
sage: from pathlib import Path
sage: input_file_name = Path("DOCTEST-AES-Models/MixColumn51845.mps")
sage: output_file_name = Path("DOCTEST-AES-Models/MixColumn51845.sol")
sage: solve(input_file_name, output_file_name, SOLVER.SCIP)

```

Notice that you do not have to keep the sage session alive while you solve the model externally. Again, this is something we have to simulate here, by simply deleting the aes and model_options objects:

```

sage: del aes
sage: del model_options

```

Next, we finish the model:

```

sage: # optional - scip
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: aes = AES_CVL(R=10)

```

(continues on next page)

(continued from previous page)

```

sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.LINEAR,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,
....:     path=Path("./DOCTEST-AES-Models/"))
sage: aes.model(model_options) # doctest: +NORMALIZE_WHITESPACE
Using existing file DOCTEST-AES-Models/MixColumn51845.sol,
make sure it is up to date!
2848 variables and 2977 constraints were written to
'DOCTEST-AES-Models/AES.mps'
Boolean Program (minimization, 2848 variables, 2977 constraints)

```

Now, you would again copy the .mps file, this time of course the AES.mps, to a machine with a supported solver. After solving, the solver will already tell you, that the objective value is 55. For generating a report, you would copy the AES.sol file to the DOCTEST-AES-Models directory. Again, we have to simulate this:

```

sage: # optional - scip
sage: from civerly.solvers import solve
sage: from pathlib import Path
sage: solve(input_file_name=Path("DOCTEST-AES-Models/AES.mps"),
....:     output_file_name=Path("DOCTEST-AES-Models/AES.sol"),
....:     solver = SOLVER.SCIP)

```

And again, we simulate restarting CiVerLy:

```

sage: # optional - scip
sage: del aes
sage: del model_options

```

Now, we first verify that the objective value is indeed 55:

```

sage: # optional - scip
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: from civerly.solvers import get_objective_value
sage: sol_file_name = Path("DOCTEST-AES-Models/AES.sol")
sage: get_objective_value(sol_file_name, SOLVER.SCIP)
55

```

Which tells us that there are indeed 55 active S-boxes. To visualize the corresponding activity pattern, we can generate a PDF report:

```

sage: from civerly.cipher_implementations.aes import AES_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: aes = AES_CVL(R=10)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.LINEAR,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,

```

(continues on next page)

(continued from previous page)

```

....:     path=Path("./DOCTEST-AES-Models/"))
sage: aes.generate_report(model_options) # optional - scip
Output file in: DOCTEST-AES-Models/AES.pdf

```

To conclude our example, we remove the generated files:

```
sage: shutil.rmtree("DOCTEST-AES-Models", ignore_errors=True)
```

```
class civerly.cipher_implementations.aes.AES_CVL(*args, **kwargs)
```

Bases: object

Implementation of the AES in CiVerLy.

`__init__(R, name=None) → None`

Implement AES in CiVerLy.

Notice that the key schedule is omitted. This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

INPUT:

- R – integer; Number of rounds.
- name – string; The name of the cipher (optional). This will be used to name the cipher and the corresponding file generated (such as the reports and cipher graphs).

EXAMPLES:

Encrypt a message (to verify the implementation):

```

sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.cipher_implementations.aes import AES_CVL
sage: aes = AES_CVL(R=4)
sage: hex(vec_to_int(aes(int_to_vec(0x12345678, 128))))
'0x1491385e17259a1555f377e76ade6090'

```

3.13.3 Ascon

```
class civerly.cipher_implementations.ascon.ASCON_CVL(*args, **kwargs)
```

Bases: object

`__init__(R=12, name=None)`

The CiVerLy implementation of ASCON. It takes the following arguments:

- R – integer; Number of rounds.
- name – string; The name of the cipher (optional). Will be used to name the cipher and the corresponding files generated (such as the reports and cipher graphs).

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

```

sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.cipher_implementations.ascon import ASCON_CVL
sage: ascon = ASCON_CVL(R=4)
sage: hex(vec_to_int(ascon(int_to_vec(0x12345678, 320))))

```

(continues on next page)

(continued from previous page)

```
↪ '0xc11ab05dd48d088bb6c89db8226db8c5320a365e071923f271052f313bf57555ec2f2a6f79ca5be0
↪ '
```

3.13.4 Chacha

class civerly.cipher_implementations.chacha.ChachaQRF_CVL(*args, **kwargs)

Bases: object

__init__(name=None)

The CiVerLy implementation of the Chacha QRF. Since there is nothing to tweak, it does not take any parameters. This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.cipher_implementations.chacha \
....:     import ChachaQRF_CVL
sage: qrf = ChachaQRF_CVL()
sage: hex(vec_to_int(qrf(int_to_vec(0x12345678, 128))))
'0x812345677611b618b1cf660b5b5753d7'
sage: hex(vec_to_int(qrf(int_to_vec(
....:     0xaeaeaea0_aeaeaea1_aeaeaea2_aeaeaea3,
....:     128))))
'0x4e209e041e6d48ca64ff9a52c26df7bd'
```

class civerly.cipher_implementations.chacha.Chacha_CVL(*args, **kwargs)

Bases: object

__init__(R=20, name=None)

The CiVerLy implementation of the Chacha QRF. It takes the following arguments:

- R – integer; Number of rounds.

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.cipher_implementations.chacha import Chacha_CVL
sage: chacha = Chacha_CVL(10)
sage: hex(vec_to_int(chacha(int_to_vec(0x12345678, 384))))
↪ '0x830ea31f9682b3636956bd110cc048c79cc417bfa8f4bf073478c1627a57a6636b5722956e674b993b8f0622d
↪ '
sage: chacha_cipher = Chacha_CVL(20)
sage: hex(vec_to_int(chacha_cipher(int_to_
↪ vec(0xaeaeaea0_aeaeaea1_aeaeaea2_aeaeaea3_aeaeaea4_aeaeaea5_aeaeaea6_aeaeaea7_
↪ aeaeaea8_aeaeaea9_aeaeaeaA_aeaeaeaB, 384))))
↪ '0x7b435b3af7b4f2ef11672cb917259cb6f218b24ca321714fe1c2dc6f628b75de65cffb65d6c83b2f029f9e0e3
↪ '
```

3.13.5 Craft

`class civerly.cipher_implementations.craft.CRAFT_CVL(*args, **kwargs)`

Bases: object

`__init__(R, name=None) → None`

The CiVerLy implementation of CRAFT. It takes the following arguments:

- `R` – integer; Specifies the number of rounds that are performed.
- `name` – string (optional); The name of the cipher. Will be used to name the cipher and the corresponding files generated (such as the reports and cipher graphs).

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

```
sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.cipher_implementations.craft import CRAFT_CVL
sage: from civerly.model_options import *
sage: craft = CRAFT_CVL(10)
sage: hex(vec_to_int(craft(int_to_vec(0x12345678,64))))
'0x5d1989d86b887cce'
```

Determine the number of active S-boxes using a word-wise model based on the the branch number:

```
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.BRANCH_NUMBER,
....:     solver=SOLVER.SCHIP,
....:     path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options) # optional - scip
5896 variables and 6121 constraints were written to
'DOCTEST-CRAFT-Models/CRAFT.mps'
10
```

Notice that the bound of 10 active S-boxes is rather trivial (one per round). Hence, this is a good example of why the more accurate modeling of the linear layer is useful. But first some cleanup:

```
sage: import shutil
sage: shutil.rmtree("DOCTEST-CRAFT-Models") # optional - scip
```

Now the improved modeling:

```
sage: craft = CRAFT_CVL(10)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.LINEAR,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.WORDWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.GENERALIZED_WORDWISE,
....:     solver=SOLVER.SCHIP,
....:     path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options) # optional - scip
5856 variables and 6041 constraints were written to
```

(continues on next page)

(continued from previous page)

```
'DOCTEST-CRAFT-Models/CRAFT.mps'
36
sage: shutil.rmtree("DOCTEST-CRAFT-Models") # optional - scip
```

Indeed, 36 active S-boxes is a much better bound.

To get even more explicit results, such as an optimal differential or linear trail through CRAFT, we can use bitwise modeling. However, this is computationally more difficult than the previous tests:

```
sage: craft = CRAFT_CVL(3)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.BITWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.CONVEX_HULL,
....:     sbbox_modeling=SBOX_MODELING.CONVEX_HULL,
....:     solver=SOLVER.SCHIP,
....:     path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options) # optional - scip
7440 variables and 9057 constraints were written to
'DOCTEST-CRAFT-Models/CRAFT.mps'
8
sage: shutil.rmtree("DOCTEST-CRAFT-Models") # optional - scip
```

Here the objective value is $-\log_2(p)$, with p being the differential probability (or respectively the linear correlation) of the found trail.

We repeat the same experiment but this time use dummy variables to model the linear layer:

```
sage: craft = CRAFT_CVL(3)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
....:     granularity=GRANULARITY.BITWISE,
....:     linear_layer_modeling=LINEAR_LAYER_MODELING.MORE_DUMMIES,
....:     sbbox_modeling=SBOX_MODELING.CONVEX_HULL,
....:     solver=SOLVER.SCHIP,
....:     path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options) # optional - scip
7728 variables and 8001 constraints were written to
'DOCTEST-CRAFT-Models/CRAFT.mps'
8
sage: shutil.rmtree("DOCTEST-CRAFT-Models") # optional - scip
```

It is also possible to use SAT to model the cipher. The results should match those that MILP-modeling produces:

```
sage: # optional - cryptominisat # optional - espresso
sage: from civerly.cipher_implementations.craft import CRAFT_CVL
sage: from civerly.model_options import *
sage: craft = CRAFT_CVL(3)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.SAT,
```

(continues on next page)

(continued from previous page)

```

....: granularity=GRANULARITY.BITWISE,
....: linear_layer_modeling=LINEAR_LAYER_MODELING.EXCLUDE_ODD,
....: sbbox_modeling=SBOX_MODELING.LOGICAL_COND_ESPRESSO,
....: solver=SOLVER.CRYPTOMINISAT,
....: path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options)
7440 variables and 17201 clauses were written to
'DOCTEST-CRAFT-Models/CRAFT.cnf'
[ 0 ,100] (trying w = 50) : SAT
[ 0 , 50] (trying w = 25) : SAT
[ 0 , 25] (trying w = 12) : SAT
[ 0 , 12] (trying w = 6) : UNSAT
[ 7 , 12] (trying w = 9) : SAT
[ 7 , 9] (trying w = 8) : SAT
[ 7 , 8] (trying w = 7) : UNSAT
8
sage: craft.generate_report(model_options)
Output file in: DOCTEST-CRAFT-Models/CRAFT.pdf
sage: trail = str(craft.get_trail(model_options))
sage: assert "Unnamed Component" not in trail

```

Now with CaDiCaL as solver:

```

sage: # optional - cadical # optional - espresso
sage: from civerly.cipher_implementations.craft import CRAFT_CVL
sage: from civerly.model_options import *
sage: craft = CRAFT_CVL(3)
sage: model_options = MODEL_OPTIONS(
....: cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....: optimization=OPTIMIZATION.SAT,
....: granularity=GRANULARITY.BITWISE,
....: linear_layer_modeling=LINEAR_LAYER_MODELING.EXCLUDE_ODD,
....: sbbox_modeling=SBOX_MODELING.LOGICAL_COND_ESPRESSO,
....: solver=SOLVER.CADICAL,
....: path=Path("./DOCTEST-CRAFT-Models/"))
sage: craft.analyse(model_options)
7440 variables and 17201 clauses were written to
'DOCTEST-CRAFT-Models/CRAFT.cnf'
[ 0 ,100] (trying w = 50) : SAT
[ 0 , 50] (trying w = 25) : SAT
[ 0 , 25] (trying w = 12) : SAT
[ 0 , 12] (trying w = 6) : UNSAT
[ 7 , 12] (trying w = 9) : SAT
[ 7 , 9] (trying w = 8) : SAT
[ 7 , 8] (trying w = 7) : UNSAT
8
sage: craft.generate_report(model_options)
Output file in: DOCTEST-CRAFT-Models/CRAFT.pdf
sage: trail = str(craft.get_trail(model_options))
sage: assert "Unnamed Component" not in trail
sage: import shutil
sage: shutil.rmtree("DOCTEST-CRAFT-Models", ignore_errors=True)

```

3.13.6 DES

```
class civerly.cipher_implementations.des.DES_F_CVL(*args, **kwargs)
```

Bases: object

```
__init__()
```

The implementation of DES-F. The test vectors are taken from <https://crypto.stackexchange.com/questions/65996/64-des-full-example-with-all-the-stages>.

```
class civerly.cipher_implementations.des.DES_CVL(*args, **kwargs)
```

Bases: object

```
__init__(R, rks=[], name=None) → None
```

The DES implementation. The test vectors are taken from <https://crypto.stackexchange.com/questions/65996/64-des-full-example-with-all-the-stages>.

3.13.7 GIFT

Rudimentary implementation of the GIFT block cipher.

We only implement the S-box and the linear layer. This is already enough to analyze GIFT with CiVerLy.

```
class civerly.cipher_implementations.gift.GIFT_CVL(*args, **kwargs)
```

Bases: object

Rudimentary implementation of GIFT.

```
__init__(R=28, name=None)
```

Initialise GIFT.

3.13.8 HALFLOOP-24

Implementation of HALFLOOP-24.

```
class civerly.cipher_implementations.halfloop.HALFLOOP_CVL(*args, **kwargs)
```

Bases: object

Implementation of HALFLOOP-24 in CiVerLy.

Warning: Notice that while the implementation already allows to model HALFLOOP and finds the probability-one differential over six rounds, it is not complete. That is, the values of some constants are missing and hence the evaluating this implementation will return wrong results.

```
__init__(R, name=None) → None
```

Implement HALFLOOP in CiVerLy.

INPUT:

- R – integer; Number of rounds.
- name – string; The name of the cipher (optional). This will be used to name the cipher and the corresponding file generated (such as the reports and cipher graphs).

3.13.9 HURDLE

`civerly.cipher_implementations.hurdle.hurdle_key_schedule(masterkey)`

The HURDLE key schedule, as implemented in *HURDLE_{s et_key}* in MidnightBlue’s implementation.

class `civerly.cipher_implementations.hurdle.HURDLE_F_CVL(*args, **kwargs)`

Bases: object

`__init__(rk=None) → None`

Implementation of HURDLE-II’s F function, imitating MidnightBlue’s implementation.

class `civerly.cipher_implementations.hurdle.HURDLE_CVL(*args, **kwargs)`

Bases: object

`__init__(R=16, k=None, name=None) → None`

Implementation of the HURDLE cipher, imitating MidnightBlue’s implementation.

3.13.10 Present

class `civerly.cipher_implementations.present.PRESENT_CVL(*args, **kwargs)`

Bases: object

`__init__(R=31, rks=[], name=None)`

The CiVerLy implementation of PRESENT. It takes in the following arguments:

- `R` – integer; Number of rounds.
- `rks` – list (optional); Specifies the roundkey values of PRESENT, in order to being able to properly test the implementation. Is required to have length $R + 1$, and defaults to `[0, ..., 0]`.
- `name` – string (optional); The name of the cipher. Will be used to name the cipher and the corresponding files generated (such as the reports and cipher graphs).

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

Encrypt a message (for verifying the implementation):

```
sage: from civerly.cipher_implementations.present \
....:     import PRESENT_CVL
sage: from civerly.util import int_to_vec, vec_to_int
sage: present_cipher = PRESENT_CVL(R=10)
sage: hex(vec_to_int(present_cipher(int_to_vec(0xabcd1234, 64))))
'0xdd9e25f5bd58fdc9'
```

Since PRESENT is a word-based cipher, we can perform branch-number-based wordwise modeling:

```
sage: from civerly.cipher_implementations.present \
....:     import PRESENT_CVL
sage: from civerly.model_options import *
sage: from pathlib import Path
sage: present_cipher = PRESENT_CVL(R=4)
sage: model_options = MODEL_OPTIONS(
....:     cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....:     optimization=OPTIMIZATION.MILP,
```

(continues on next page)

(continued from previous page)

```

....: granularity=GRANULARITY.WORDWISE,
....: linear_layer_modeling=LINEAR_LAYER_MODELING.BRANCH_NUMBER,
....: solver=SOLVER.SCHIP,
....: path=Path("./DOCTEST-PRESENT-Models/")
sage: present_cipher.analyse(model_options) # optional - scip
1284 variables and 1341 constraints were written to
'DOCTEST-PRESENT-Models/PRESENT.mps'
4

```

Clean-up of the generated files:

```

sage: import shutil
sage: shutil.rmtree("DOCTEST-PRESENT-Models") # optional - scip

```

Of course, since the branch number of any word-permutation is 2, this result is not very interesting and unprecise, as the optimal solution here would be one active word per round, which is specifically avoided to be possible in PRESENT. This indicates that generalized wordwise modeling might be a more reasonable approach. However, performing generalized wordwise modeling is extremely costly since we would need to iterate over 2^{2n} possible transition, for a linear layer transforming n words. For $n = 16$, as is the case for PRESENT, this results in an infeasibly complex task, which takes an unproportionally long time to perform. Note that this is due to the fact that generalized wordwise modeling ignores the fact that this current linear layer is actually a word-permutation, which would, in theory, be significantly easier to model. In turn, this also means that the complexity of modeling this permutation using generalized wordwise modeling is equal to the complexity of modeling **any** linear layer acting on 16 words.

However, the bitwise modeling technique does not come with such issues and is also more precise, since the result will be an actually possible differential (or linear) trail instead of an activity pattern of the words. As the linear layer of cipher{PRESENT} is a bit permutation and therefore trivial to model, we do not need to specify a linear layer modeling technique:

```

sage: from civerly.cipher_implementations.present \
....: import PRESENT_CVL
sage: from civerly.model_options import *
sage: present_cipher = PRESENT_CVL(R=4)
sage: model_options = MODEL_OPTIONS(
....: cryptanalysis=CRYPTANALYSIS.DIFFERENTIAL,
....: optimization=OPTIMIZATION.MILP,
....: granularity=GRANULARITY.BITWISE,
....: sbbox_modeling=SBOX_MODELING.CONVEX_HULL,
....: solver=SOLVER.SCHIP,
....: path=Path("./DOCTEST-PRESENT-Models/"))
sage: present_cipher.analyse(model_options) # optional - scip
5312 variables and 6081 constraints were written to
'DOCTEST-PRESENT-Models/PRESENT.mps'
12
sage: import shutil
sage: shutil.rmtree("DOCTEST-PRESENT-Models") # optional - scip

```

Here, the analysis output 12 means that the best differential trail over 4 rounds of PRESENT has a probability of 2^{-12} .

3.13.11 Simon

`class civerly.cipher_implementations.simon.SIMON_CVL(*args, **kwargs)`

Bases: object

`__init__(block_size, key_size, R=None, rks=[], use_rotand=True, name=None)`

The CiVerLy implementation of SIMON. It takes the following arguments:

- `block_size`— integer; Determines the block size of the SIMON instance. Together with `key_size`, it determines the SIMON-2n-mn instance.
- `key_size`— integer; Determines the key size of the SIMON instance. Together with `block_size`, it determines the SIMON-2n-mn instance.
- `rks` — list (optional); Specifies the roundkey values of SIMON, in order to being able to properly test the implementation. Is required to have length $R+1$, and defaults to `[0, ..., 0]`.
- `name` — string (optional); The name of the cipher. Will be used to name the cipher and the corresponding files generated (such as the reports and cipher graphs).

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported. SIMON can be implemented in two different ways, either using separate *RotateLayer_{CVL}* and *AND_{CVL}* components, or by combining them into a dedicated *ROT_AND_CVL* component. The second option allows for more precise results when modeling, as the modeling methods from <https://eprint.iacr.org/2015/145.pdf> are used instead of treating the inputs as independent.

EXAMPLES:

```
sage: from civerly.util import int_to_vec, vec_to_int
sage: from civerly.cipher_implementations.simon import SIMON_CVL
sage: simon_cipher = SIMON_CVL(64,128)
sage: hex(vec_to_int(simon_cipher(int_to_vec(0xabcd1234, 64))))
'0xa26a16020880adbd'
```

3.13.12 SKINNY

`class civerly.cipher_implementations.skinny.SKINNY_CVL(*args, **kwargs)`

Bases: object

`consts = [1, 3, 7, 15, 31, 62, 61, 59, 55, 47, 30, 60, 57, 51, 39, 14, 29, 58, 53, 43, 22, 44, 24, 48, 33, 2, 5, 11, 23, 46, 28, 56, 49, 35, 6, 13, 27, 54, 45, 26, 52, 41, 18, 36, 8, 17, 34, 4, 9, 19, 38, 12, 25, 50, 37, 10, 21, 42, 20, 40, 16, 32]`

`create_tk_schedules(z)`

Helper function, creating the Tweakkey schedules. These are AESlike ciphers because of the LFSR layer not obeying the “MixColumn-condition” for AESlike ciphers.

The reference values below are taken from the SKINNY implementation in (<https://github.com/hadipourh/skinny>). Note that these also align with the test vectors from the SKINNY paper (<https://eprint.iacr.org/2016/660.pdf>).

`__init__(n=64, t=64, R=None, key=None, name=None)`

The civerly implementation of SKINNY. It takes the following arguments:

- `n` — integer; The block size of SKINNY. It is required that $n \in \{64, 128\}$.
- `t` — integer; The tweakkey size of SKINNY. Needs to fulfill $t \in \{n, 2n, 3n\}$.

- **key** – integer (optional); The (tweak-)key value for SKINNY. If no key is specified, it is defaulted to 0.
- **name** – string (optional); The name of the SKINNY cipher. Is defaulted to “SKINNY”.

The test vectors used below are taken from SKINNY’s original specification (<https://eprint.iacr.org/2016/660.pdf>):

```
sage: from civerly.cipher_implementations.skinny import SKINNY_CVL sage: from civerly.util import int_to_vec, vec_to_int sage: skinny = SKINNY_CVL( ....: 64, 64, key=0xf5269826fc681238, name="SKINNY-64-64") sage: vec_to_int(skinny(int_to_vec(0x06034f957724d19d, 64))) == ....: 0xbb39dfb2429b8ac7 True sage: skinny = SKINNY_CVL( ....: 64, 128, key=0x9eb93640d088da63_76a39d1c8bea71e1, ....: name="SKINNY-64-128") sage: vec_to_int(skinny(int_to_vec(0xcf16cfe8fd0f98aa, 64))) == ....: 0x6ceda1f43de92b9e True sage: skinny = SKINNY_CVL( ....: 64, 192, ....: key=0xed00c85b120d6861_8753e24bfd908f60_b2dbb41b422dfcd0, ....: name="SKINNY-64-192") sage: vec_to_int(skinny(int_to_vec(0x530c61d35e8663c3, 64))) == ....: 0xdd2cf1a8f330303c True sage: skinny = SKINNY_CVL(128, 128, ....: key=0x4f55cfb0520cac52fd92c15f37073e93, name="SKINNY-128-128") sage: vec_to_int(skinny(int_to_vec( ....: 0xf20adb0eb08b648a3b2eed1f0adda14, 128))) == ....: 0x22ff30d498ea62d7e45b476e33675b74 True sage: skinny = SKINNY_CVL(128, 256, ....: key=0x009cec81605d4ac1d2ae9e3085d7a1f3_1ac123ebfc00fddcf01046ceeddfcab3, ....: name="SKINNY-128-256") sage: vec_to_int(skinny(int_to_vec( ....: 0x3a0c47767a26a68dd382a695e7022e25, 128))) == ....: 0xb731d98a4bde147a7ed4a6f16b9b587f True sage: skinny = SKINNY_CVL(128, 384, ....: key=0xdf889548cfc7ea52d296339301797449_ab588a34a47f1ab2dfe9c8293f1, ....: name="SKINNY-128-384") sage: vec_to_int(skinny(int_to_vec( ....: 0xa3994b66ad85a3459f44e92b08f550cb, 128))) == ....: 0x94ecf589e2017c601b38c6346a10dcfa True
```

3.13.13 Speck

class `civerly.cipher_implementations.speck.SPECK_CVL(*args, **kwargs)`

Bases: `object`

__init__(*block_size*, *key_size*, *R=None*, *rks=[]*, *name=None*)

The CiVerLy implementation of SPECK. It takes the following arguments:

- **block_size**– integer; Determines the block size of the SPECK instance. Together with **key_size**, it determines the SPECK-2n-mn instance
- **key_size**– integer; Determines the key size of the SPECK instance. Together with **block_size**, it determines the SPECK-2n-mn instance
- **rks** – list (optional); Specifies the roundkey values of SPECK, in order to being able to properly test the implementation. Is required to have length R+1, and defaults to [0, ..., 0].
- **name** – string (optional); The name of the cipher. Will be used to name the cipher and the corresponding files generated (such as the reports and cipher graphs).

This cipher is “plug-and-play” usable, i.e. it can be directly used when imported.

EXAMPLES:

```
sage: from civerly.util import vec_to_int, int_to_vec
sage: from civerly.cipher_implementations.speck import SPECK_CVL
sage: speck_cipher = SPECK_CVL(96, 144)
sage: hex(vec_to_int(speck_cipher(int_to_vec(0xabcd1234, 96))))
'0xef12cffdd86766108a1af809'
```

DEVELOPER MANUAL

CiVerLy is implemented as a Python package extending the computer algebra system SageMath, which we often refer to as `sage`. As such, it can be installed using the Python package manager `pip`, which comes with `sage`, from the provided source files. Notice that this installation does not inherently require an internet connection, as `pip` can be instructed to work offline. Of course, the source code of CiVerLy as well as the optional but highly recommended external dependencies such as the Espresso minimizer or the MILP and SAT solvers must be obtained somehow which usually involves an internet connection. Further, CiVerLy being a Python package also means that, in contrast to the initial mock-ups, CiVerLy will not be used as a standalone tool but within `sage`, either in a `.sage` script or in an interactive `sage` session.

4.1 Files of CiVerLy

Here, we briefly discuss the different files that together make up CiVerLy.

4.1.1 Overview

```
civerly/
├── src/
│   └── civerly/
│       ├── __init__.py
│       ├── component.py
│       ├── cipher.py
│       ├── sbboxcipher.py
│       ├── wordsboxcipher.py
│       ├── aeslike.py
│       ├── wordbasedcipher.py
│       ├── addrx.py
│       ├── andrx.py
│       ├── model_options.py
│       ├── solvers.py
│       ├── util.py
│       ├── benchmark.py
│       └── cipher_implementations/
│           ├── aes.py
│           └── ... (other files)
├── docs/
└── Makefile
```

(continues on next page)

(continued from previous page)

```
├── source/
│   └── ... (other files)
├── pyproject.toml
├── setup.py
└── MANIFEST.in
```

4.1.2 Source Files

Most importantly, there are the source files in `src/civerly`. Many of those simply implement the class hierarchy for CiVerLy ciphers. `component.py` implements all components that are available in CiVerLy and also contains the code to model each of these. `model_options.py` collects all the available options for modeling. `solvers.py` contains code to interact with external MILP and SAT solvers. `util.py` contains other utility functions and `benchmark.py` contains code to generate benchmarks, i.e., timings for the generation and solving of models. `cipher_implementations` contains the implementation of some well-known ciphers and also examples how these can be modeled with CiVerLy.

4.1.3 Doc Files

`source` contains the source files for this document and `Makefile` provides the commands to build it.

4.1.4 Project Files

`pyproject.toml`, `setup.py` and `MANIFEST.in` contain some information on the CiVerLy (Python) project, e.g., its dependencies.

4.2 The Documentation

We build the documentation of CiVerLy (this document) using [Sphinx](#). That is, we essentially mimic what `sage` is doing as well. The actual source code is documented using so called docstrings at the beginning of a class or function. These are augmented by additional documentation which is stored in `.rst` files in the aforementioned `docs/source/` directory. Sphinx then processes these files and the docstrings to build the documentation. Notice that Sphinx already comes with `sage` and hence is no additional dependency.

The available formats for the documentation are HTML and PDF. The HTML version can either be used by opening the corresponding `.html` files locally in a web browser or by making them available on a web server. As we expect the documentation to usually be not consumed in a read beginning-to-end form, we recommend to use the HTML version for day-to-day use and the PDF version for archiving.

4.2.1 Rebuilding the Documentation

The documentation of CiVerLy (this document) is build using Sphinx. To modify it, either edit the docstrings in the Python source files of CiVerLy or edit the files in `civerly/docs/source`. In case you modified the Python sources, you first have to reinstall CiVerLy by running

```
$ sage --python -m pip install --no-index --no-build-isolation .
```

in the `civerly` directory. Now, change the directory to the `civerly/docs` directory. Then, to build the HTML documentation run:


```
$ sage -sh -c "make html"
```

To build the PDF version of the documentation run:

```
$ sage -sh -c "make pdf"
```

4.3 Automated Tests

Naturally, we also implemented automated testing which is a indispensable foothold of modern software development. On the one hand, these ensure that our models actually lead to the correct results. On the other hand, they also make it easy to detect if further additions to the CiVerLy code break any functionality that was working correctly before. Again, we mimic what sage does. That is, the aforementioned docstrings can contain **EXAMPLES** and **TESTS** blocks. These are made up of sage commands and their expected outputs. This already is quite a useful addition to the documentation as examples often deepen the understanding of a function but sage can also automatically evaluate all those examples and verify that the computed output is indeed the output that was put in the docstring.

4.3.1 Tests involving external dependencies

For obvious reasons, our tests must involve external tools like MILP and SAT solvers. This however comes with some pitfalls. For instance, consider a developer that wants to run all CiVerLy tests but who has not installed all solvers used by CiVerLy. Luckily, sage provides a solution for this as test can be marked as optional. The implementations of ciphers in CiVerLy commonly feature tests like the following.

```
sage: aes.analyse(model_options) # optional - glpk
```

This test is then only run when the sage -t invocation contains the optional glpk flag. This in turn introduces a new pitfall. A developer that forgets to put this optional flag will not notice that it is missing if the solver is installed. This then makes the test fail for other developers that do not have the solver installed. To prevent this scenario, we added some functionality to CiVerLy that can detect such missing optional flags. In a nutshell, before invoking a solver CiVerLy checks if this solver was marked as disabled in a Linux environment variable. If this is the case, CiVerLy stops with an error. More concretely,

```
env CIVERLY_DISABLE_GUROBI=1 sage -t --optional=sage,scip,glpk,cryptominisat,cadical,
↳ espresso civerly/src/civerly
```

will run all tests in CiVerLy except for those that require Gurobi. If the tests invoke Gurobi anyway, CiVerLy will stop with an error.

4.3.2 Running tests

As the example above already shows, tests are executed using sage -t. Extensive documentation for the sage doctest framework is available [online](#). Here, we want to highlight some useful options. First, we can mark doctests that will take long time as follows:

```
sage: present_cipher.analyse(model_options) # long
```

To include these when running the tests, add --long to the sage -t command.

Further, sage can run tests in parallel. To do so, add for instance --nthreads=8 to the sage -t command. When doing so, ensure that the different tests do not access the same files.

Another useful flag is `--exitfirst` which will stop all tests as soon as one failed. Use `--logfile=filename.log` to write a log file. Finally, we want to notice that `sage -t` can also be executed on the `.rst` files of the documentation. Do so to ensure correctness of examples in the documentation.

PYTHON MODULE INDEX

C

- civerly.addr, 48
- civerly.aeslike, 46
- civerly.andrx, 48
- civerly.benchmark, 53
- civerly.cipher, 39
- civerly.cipher_implementations.abc, 53
- civerly.cipher_implementations.aes, 54
- civerly.cipher_implementations.ascon, 57
- civerly.cipher_implementations.chacha, 58
- civerly.cipher_implementations.craft, 59
- civerly.cipher_implementations.des, 62
- civerly.cipher_implementations.gift, 62
- civerly.cipher_implementations.halfloop, 62
- civerly.cipher_implementations.hurdle, 63
- civerly.cipher_implementations.present, 63
- civerly.cipher_implementations.simon, 65
- civerly.cipher_implementations.skinny, 65
- civerly.cipher_implementations.speck, 66
- civerly.model_options, 27
- civerly.sboxcipher, 44
- civerly.solvers, 49
- civerly.util, 51
- civerly.wordbasedcipher, 45
- civerly.wordsboxcipher, 46

Symbols

- `__call__()` (civerly.cipher.Cipher method), 41
 - `__init__()` (civerly.aeslike.AESlike method), 47
 - `__init__()` (civerly.cipher.Cipher method), 40
 - `__init__()` (civerly.cipher.CipherNotValidException method), 39
 - `__init__()` (civerly.cipher_implementations.abc.ABC_CVLE method), 53
 - `__init__()` (civerly.cipher_implementations.aes.AES_CVLE method), 57
 - `__init__()` (civerly.cipher_implementations.ascon.ASCON_CVLE method), 57
 - `__init__()` (civerly.cipher_implementations.chacha.Chacha_CVLE method), 58
 - `__init__()` (civerly.cipher_implementations.chacha.Chacha_CVLE method), 58
 - `__init__()` (civerly.cipher_implementations.craft.CRAFT_CVLE method), 59
 - `__init__()` (civerly.cipher_implementations.des.DES_CVLE method), 62
 - `__init__()` (civerly.cipher_implementations.des.DES_F_CVLE method), 62
 - `__init__()` (civerly.cipher_implementations.gift.GIFT_CVLE method), 62
 - `__init__()` (civerly.cipher_implementations.halfloop.HALFLOOP_CVLE method), 62
 - `__init__()` (civerly.cipher_implementations.hurdle.HURDLE_CVLE method), 63
 - `__init__()` (civerly.cipher_implementations.hurdle.HURDLE_F_CVLE method), 63
 - `__init__()` (civerly.cipher_implementations.present.PRESENT_CVLE method), 63
 - `__init__()` (civerly.cipher_implementations.simon.SIMON_CVLE method), 65
 - `__init__()` (civerly.cipher_implementations.skinny.SKINNY_CVLE method), 65
 - `__init__()` (civerly.cipher_implementations.speck.SPECK_CVLE method), 66
 - `__init__()` (civerly.component.Component method), 32
 - `__init__()` (civerly.sboxcipher.SBoxCipher method), 44
 - `__init__()` (civerly.wordbasedcipher.WordBasedCipher method), 45
 - `__parse_cadical()` (in module civerly.solvers), 50
 - `__parse_cryptominisat()` (in module civerly.solvers), 50
 - `_copy_over_dictionaries_recursively()` (civerly.component.Component method), 32
 - `finish_milp()` (civerly.sboxcipher.SBoxCipher method), 44
 - `float_or_int()` (in module civerly.solvers), 50
 - `_generate_constraints_sum_leq_int_LS24()` (in module civerly.util), 52
 - `_model_milp()` (civerly.sboxcipher.SBoxCipher method), 44
 - `_process_solution_file()` (in module civerly.solvers), 50
 - `_sat_get_clauses_espresso()` (in module civerly.util), 52
- ## A
- ABC_CVLE (class in civerly.cipher_implementations.abc), 53
 - add_output() (civerly.cipher.Cipher method), 42
 - add_subcipher() (civerly.aeslike.AESlike method), 47
 - add_subcipher() (civerly.cipher.Cipher method), 40
 - add_subcipher() (civerly.wordbasedcipher.WordBasedCipher method), 45
 - AddrX (class in civerly.addr), 48
 - AES_CVLE (class in civerly.cipher_implementations.aes), 57
 - AESlike (class in civerly.aeslike), 47
 - AND_CVLE (civerly.cipher.Cipher method), 43
 - AND_CVLE (class in civerly.component), 35
 - Andrx (class in civerly.andrx), 48
 - ASCON_CVLE (class in civerly.cipher_implementations.ascon), 57
- ## B
- benchmark() (in module civerly.benchmark), 53
 - BITWISE (civerly.model_options.GRANULARITY attribute), 27
 - BRANCH_NUMBER (civerly.model_options.LINEAR_LAYER_MODELING attribute), 28

branch_number_differential
 (*civerly.component.LinearLayer_CVL* *prop-*
 erty), 36

branch_number_linear
 (*civerly.component.LinearLayer_CVL* *prop-*
 erty), 37

C

C_CVL (*class in civerly.component*), 32

CADICAL (*civerly.model_options.SOLVER* *attribute*), 29

Chacha_CVL (*class in civerly.cipher_implementations.chacha*),
 58

ChachaQRF_CVL (*class in civerly.cipher_implementations.chacha*),
 58

Cipher (*class in civerly.cipher*), 40

CipherNotValidException, 39

civerly.addrx
 module, 48

civerly.aeslike
 module, 46

civerly.andrx
 module, 48

civerly.benchmark
 module, 53

civerly.cipher
 module, 39

civerly.cipher_implementations.abc
 module, 53

civerly.cipher_implementations.aes
 module, 54

civerly.cipher_implementations.ascon
 module, 57

civerly.cipher_implementations.chacha
 module, 58

civerly.cipher_implementations.craft
 module, 59

civerly.cipher_implementations.des
 module, 62

civerly.cipher_implementations.gift
 module, 62

civerly.cipher_implementations.halfloop
 module, 62

civerly.cipher_implementations.hurdle
 module, 63

civerly.cipher_implementations.present
 module, 63

civerly.cipher_implementations.simon
 module, 65

civerly.cipher_implementations.skinny
 module, 65

civerly.cipher_implementations.speck
 module, 66

civerly.model_options
 module, 27

civerly.sboxcipher
 module, 44

civerly.solvers
 module, 49

civerly.util
 module, 51

civerly.wordbasedcipher
 module, 45

civerly.wordsboxcipher
 module, 46

cols (*civerly.aeslike.AESlike* *property*), 47

Component (*class in civerly.component*), 31

consts (*civerly.cipher_implementations.skinny.SKINNY_CVL*
 attribute), 65

ConstXOR_CVL (*class in civerly.component*), 33

CONVEX_HULL (*civerly.model_options.LINEAR_LAYER_MODELING*
 attribute), 28

CONVEX_HULL (*civerly.model_options.SBOX_MODELING*
 attribute), 28

CRAFT_CVL (*class in civerly.cipher_implementations.craft*),
 59

create_tk_schedules()
 (*civerly.cipher_implementations.skinny.SKINNY_CVL*
 method), 65

cryptanalysis (*civerly.model_options.MODEL_OPTIONS*
 attribute), 30

CRYPTANALYSIS (*class in civerly.model_options*), 27

CRYPTOMINISAT (*civerly.model_options.SOLVER* *at-*
 tribute), 29

D

DES_CVL (*class in civerly.cipher_implementations.des*),
 62

DES_F_CVL (*class in civerly.cipher_implementations.des*),
 62

DIFFERENTIAL (*civerly.model_options.CRYPTANALYSIS*
 attribute), 27

DISTORTED_BALL (*civerly.model_options.SBOX_MODELING*
 attribute), 28

E

ERROR (*civerly.solvers.SOLVING_STATUS* *attribute*), 49

eval() (*civerly.cipher.Cipher* *method*), 41

eval() (*civerly.component.I_CVL* *method*), 32

EXCLUDE_ODD (*civerly.model_options.LINEAR_LAYER_MODELING*
 attribute), 28

G

GENERALIZED_WORDWISE
 (*civerly.model_options.LINEAR_LAYER_MODELING*
 attribute), 28

generate_report() (*civerly.cipher.Cipher* *method*), 43

- `get_objective_bounds()` (in module *civerly.solvers*), 50
- `get_objective_value()` (in module *civerly.solvers*), 50
- `get_trail()` (*civerly.cipher.Cipher* method), 43
- `GIFT_CVL` (class in *civerly.cipher_implementations.gift*), 62
- `GLPK` (*civerly.model_options.SOLVER* attribute), 29
- `GRANULARITY` (class in *civerly.model_options*), 27
- `GUROBI` (*civerly.model_options.SOLVER* attribute), 29
- ## H
- `HALFLOOP_CVL` (class in *civerly.cipher_implementations.halfloop*), 62
- `HURDLE_CVL` (class in *civerly.cipher_implementations.hurdle*), 63
- `HURDLE_F_CVL` (class in *civerly.cipher_implementations.hurdle*), 63
- `hurdle_key_schedule()` (in module *civerly.cipher_implementations.hurdle*), 63
- ## I
- `I_CVL` (class in *civerly.component*), 32
- `int_to_vec()` (in module *civerly.util*), 51
- `InvalidModelOptionException`, 31
- `is_valid` (*civerly.cipher.Cipher* property), 40
- ## L
- `LINEAR` (*civerly.model_options.CRYPTANALYSIS* attribute), 27
- `LINEAR_LAYER_MODELING` (class in *civerly.model_options*), 27
- `LinearLayer_CVL` (class in *civerly.component*), 36
- `LOGICAL_COND` (*civerly.model_options.SBOX_MODELING* attribute), 28
- `LOGICAL_COND_ESPRESSO` (*civerly.model_options.SBOX_MODELING* attribute), 28
- ## M
- `MILP` (*civerly.model_options.OPTIMIZATION* attribute), 27
- `ModAdd_CVL` (class in *civerly.component*), 35
- `model()` (*civerly.cipher.Cipher* method), 43
- `model()` (*civerly.component.Component* method), 32
- `MODEL_OPTIONS` (class in *civerly.model_options*), 29
- module
- civerly.addrx*, 48
 - civerly.aeslike*, 46
 - civerly.andrx*, 48
 - civerly.benchmark*, 53
 - civerly.cipher*, 39
 - civerly.cipher_implementations.abc*, 53
 - civerly.cipher_implementations.aes*, 54
 - civerly.cipher_implementations.ascon*, 57
 - civerly.cipher_implementations.chacha*, 58
 - civerly.cipher_implementations.craft*, 59
 - civerly.cipher_implementations.des*, 62
 - civerly.cipher_implementations.gift*, 62
 - civerly.cipher_implementations.halfloop*, 62
 - civerly.cipher_implementations.hurdle*, 63
 - civerly.cipher_implementations.present*, 63
 - civerly.cipher_implementations.simon*, 65
 - civerly.cipher_implementations.skinny*, 65
 - civerly.cipher_implementations.speck*, 66
 - civerly.model_options*, 27
 - civerly.sboxcipher*, 44
 - civerly.solvers*, 49
 - civerly.util*, 51
 - civerly.wordbasedcipher*, 45
 - civerly.wordsboxcipher*, 46
- `MORE_DUMMIES` (*civerly.model_options.LINEAR_LAYER_MODELING* attribute), 28
- ## N
- `NoSolverWarning`, 30
- ## O
- `OPTIMIZATION` (class in *civerly.model_options*), 27
- `optimize_sat()` (in module *civerly.solvers*), 49
- `OUT` (*civerly.cipher.Cipher* property), 40
- ## P
- `PermuteLayer_CVL` (class in *civerly.component*), 37
- `PRESENT_CVL` (class in *civerly.cipher_implementations.present*), 63
- ## R
- `r` (*civerly.component.ROT_AND_CVL* property), 39
- `reduction_algorithm_ST17()` (in module *civerly.util*), 52
- `RK_CVL` (class in *civerly.component*), 33
- `ROT_AND_CVL` (class in *civerly.component*), 39
- `RotateLayer_CVL` (class in *civerly.component*), 37
- `RoundkeyXOR_CVL` (class in *civerly.component*), 34
- `rows` (*civerly.aeslike.AESlike* property), 47
- ## S
- `SAT` (*civerly.model_options.OPTIMIZATION* attribute), 27
- `sat_precision` (*civerly.model_options.MODEL_OPTIONS* attribute), 30

SBox_CVL (*class in civerly.component*), 38
SBOX_MODELING (*class in civerly.model_options*), 28
SBoxCipher (*class in civerly.sboxcipher*), 44
SCIP (*civerly.model_options.SOLVER attribute*), 29
SIMON_CVL (*class in civerly.cipher_implementations.simon*),
65
SKINNY_CVL (*class in civerly.cipher_implementations.skinny*),
65
solve() (*in module civerly.solvers*), 49
solve_range (*civerly.model_options.MODEL_OPTIONS
attribute*), 30
SOLVER (*class in civerly.model_options*), 28
SOLVING_STATUS (*class in civerly.solvers*), 49
SPECK_CVL (*class in civerly.cipher_implementations.speck*),
66
SUCCESS (*civerly.solvers.SOLVING_STATUS attribute*),
49

T

TIMEOUT (*civerly.solvers.SOLVING_STATUS attribute*),
49
translate_milp_constraint() (*in module
civerly.util*), 52
translate_sat_clause() (*in module civerly.util*), 52

V

vec_to_int() (*in module civerly.util*), 51

W

WordBasedCipher (*class in civerly.wordbasedcipher*),
45
WordSBoxCipher (*class in civerly.wordsboxcipher*), 46
wordsize (*civerly.wordbasedcipher.WordBasedCipher
property*), 45
WORDWISE (*civerly.model_options.GRANULARITY
attribute*), 27

X

XOR_CVL (*class in civerly.component*), 34